University of Amsterdam
TODO: programme name (e.g. Master Software Engineering)

Measuring Placement-Sensitive Resilience under Chaos: A Pre-Registered, Layered Study in Kubernetes

Yvo Hu

MSc Thesis, University of Amsterdam

Supervisor: TODO: supervisor name
Examiner: TODO: second reader / examiner

TODO: defense date

Abstract

Kubernetes offers rich placement controls and observability, yet it remains unclear when pod placement materially affects resilience under chaos, and when aggregate resilience scores obscure that effect. This thesis presents **ChaosProbe**, a pre-registered, provenance-gated chaos-evaluation framework. It manipulates placement through two orthogonal knobs — the cross-node dependency-edge fraction f (packing to spreading) and the replication degree r (packed vs anti-affine) — plus two interventional arms (a NodeLocal DNSCache toggle, executed, and the kube-proxy mode, de-scoped to ipvs and not run), and reads every outcome simultaneously at three layers: the aggregate resilience score, the kernel/network mechanism, and the user-visible route. The study runs on the Online Boutique benchmark (ten polyglot gRPC microservices plus Redis) on a pinned cluster of eight 2-vCPU/4-GiB workers (Kubernetes v1.28.6, kube-proxy ipvs).

The evidence is three pre-registered campaigns under churn (`pod-delete`) and node failure (`node-drain`): C1 (eight complete-block dose-response sessions), C2 (24 node-drain replication-rescue sessions), and C3 (14 placement-and-DNS sessions). The hypotheses, the smallest effect sizes of interest, the per-cell sample sizes, and the analysis code were frozen before any confirmatory data was collected, and the four confirmatory hypotheses are Holm-corrected as one family.

The central finding is a real mechanism whose reach is rigorously bounded. Placement reproducibly moves a UDP-conntrack reconvergence signature — individually significant in every campaign, and the one scorecard layer with near-perfect test-retest reliability (mechanism ICC 0.994 versus a naive aggregate's 0.066) — and a NodeLocal DNSCache intervention removes it under both placements. But no confirmatory hypothesis is supported at its registered bar: the east-west dose-response is statistically detectable yet only +13.35 % across the full cross-node range, below the 15 % SESOI (H1); placement-dependence of the conntrack drop is significant but in the *opposite* direction — packing concentrates more per-node churn than spreading — so the registered conjunction fails (H2); anti-affine replication fully rescues the user-route error rate under node failure ($0.632 \rightarrow 0.0$), but the co-primary trough-depth criterion could not adjudicate rescue on this placement (H3); and the scorecard's availability layer is unreliable under churn (ICC $0.180 < 0.5$) (H5). The latency↔availability frontier the design sought is correspondingly not realizable from these data, because `pod-delete` varies the latency face but not the availability face (H4).

This last result exposed a **construction limitation in the design itself**: at a single replica, `pod-delete` cannot move availability. Reported as an *in-development, iterative design study*, the design was therefore **modified** — the availability tests were recomputed under `node-drain`, whose blast radius is placement-dependent — and the corrected design resolves the limitation exploratorily: placement governs the blast radius strongly and monotonically (all app endpoints lost under packing, roughly a third under spreading), and the latency↔availability trade-off the original frontier could not express reappears. The original iteration's valid findings (the latency dose-response and the conntrack mechanism) stand and are reported in full; the construction-limited availability results are reported *with* their limitation, not removed; and a pre-registered confirmatory re-run of the corrected design is underway.

Placement moves the kernel/network mechanism; at the pre-registered bar it does not move the user-visible placement, availability, and dose-response outcomes the hypotheses predicted. The thesis contributes a reproducible, pre-registered measurement methodology that surfaces and corrects its own construction limitations, a positively-established but precisely-bounded mechanism, and a freeze-before-analysis, provenance-gated campaign protocol in which every quoted number traces to an archived, hash-stamped run. The methodological point is that

chaos evaluation of placement needs layered measurement, pre-registration, and provenance-gated replication, not a single score: each fault class leaves its placement signal in a specific layer, and a one-number instrument's availability component is unreliable or undefined exactly where it matters.

Contents

List of Figures

Chapter 1

Introduction

1.1 Problem

The placement literature says **where pods land matters**: locality lowers inter-service latency (NetMARKS, TraDE), interference makes co-location risky (Bubble-Up, Quasar), and schedulers from Borg to Medea encode spread- and packing heuristics precisely because topology is assumed to drive performance and availability. Chaos-engineering practice, meanwhile, evaluates resilience by injecting faults and **collapsing the outcome into an aggregate score** — a single number per experiment (cf. the CSUR multi-vocal review: outcomes are defined solely against steady-state user-visible indicators).

These two bodies of practice have not been connected: nobody has measured *at which layer* a placement effect appears under a given fault class, whether it reaches the user-visible outcome, and whether an aggregate score can see it at all. If placement moves a kernel-level mechanism but not the user-visible outcome, a single score is the wrong instrument, and placement advice derived from it is unreliable.

Consider the concrete decision this thesis instruments. An operator runs a ten-service microservice storefront on a small Kubernetes cluster and must choose how to place it: pack the services onto few nodes, or spread them across many. The packing intuition comes from the network-aware scheduling literature — co-located services keep their calls node-local, and schedulers built on that observation report response-time reductions of up to 37 % [Wojciechowski et al., 2021]. The spreading intuition comes from the availability literature — services in distinct failure domains survive the loss of any one of them [Garefalakis et al., 2018] — and from the interference literature, which warns that co-location invites resource contention [Mars et al., 2011, Delimitrou and Kozyrakis, 2014].

What does today's tooling tell this operator? The Kubernetes scheduler scores nodes on resource fit, not fault isolation — a design inherited from Borg [Verma et al., 2015, Burns et al., 2016] — so the default placement answers a different question than the operator's. Chaos-engineering tools answer the resilience question directly, but with one number: inject a fault, probe the steady state, emit a score. Whether that score can actually *discriminate* between placements — whether its between-condition signal exceeds its run-to-run noise — is, to our knowledge, unexamined; this thesis audits a layered scorecard's reliability for exactly that task and finds the availability layer insufficient in the regime studied (H5, §5.5).

1.2 Research question

> **Under which chaos fault classes does pod placement measurably affect mechanism-level behaviour and user-visible outcomes in a Kubernetes microservice application, and when do aggregate resilience scores obscure those effects?**

This is framed as a **fault-class-by-measurement-layer** study, not a placement ranking and not a refutation of the placement literature. A placement effect can appear at (a) the resilience-score layer, (b) a kernel/network mechanism layer, and/or (c) the user-visible layer, and these layers need not agree — so the contribution is establishing *at which layer* an effect appears under a given fault class, and whether it reaches the user. To answer it without the freedom to choose tests after seeing the data, the study is **pre-registered**: the hypotheses, the smallest effect sizes of interest, the per-cell sample sizes, and the analysis code were frozen before any confirmatory data was collected (§4.3).

The question is decomposed into a pre-registered confirmatory family plus a descriptive deliverable, each a falsifiable target:

- **Does placement dose the mechanism's latency face?** Whether the east-west tail rises monotonically in the cross-node fraction (H1, §5.1).
- **Is the conntrack mechanism placement-dependent, and does a DNS intervention explain it?** (H2, §5.2.)
- **Does replication rescue availability under node failure, and only when replicas are spread?** (H3, §5.3.)
- **Does a layered scorecard measure resilience more reliably than a single aggregate score?** (H5, §5.5.)
- Descriptively, **what latency↔availability trade-off does the placement frontier reveal?** (H4, §5.4.)

The reliability question (H5) is logically prior: if a single score could rank placements reliably, the layered measurement design would be a refinement. Because its availability layer cannot in this regime, the layered design is what produces the findings at all.

The study at a glance: placement is manipulated through two orthogonal knobs — the cross-node dependency-edge fraction f (packing to spreading) and the replication degree r (packed vs anti-affine) — plus two interventional arms (a NodeLocal DNSCache toggle, executed, and the kube-proxy mode, de-scoped to ipvs and not run), realized by deployment-level mutation on the Online Boutique benchmark (ten polyglot gRPC microservices plus Redis) on a pinned cluster of eight 2-vCPU/4-GiB workers (Kubernetes v1.28.6, kube-proxy in ipvs mode). The evidence is three pre-registered campaigns — C1 (eight complete-block `pod-delete` dose sessions), C2 (24 `node-drain` sessions), C3 (14 `pod-delete` cache-off/on sessions). Every quoted number traces to an archived run (Appendix A), and the single-replica baseline — which makes `pod-delete` a pure churn fault — is carried explicitly through every claim (Chapter 7).

This thesis is reported as an in-development, iterative design study. A first design iteration surfaced a construction limitation on the availability axis: at a single replica, `pod-delete` produces a total but placement-independent outage, so the availability tests (H3, H4, and H5's availability sub-score) cannot move regardless of placement. The design was **modified in response to this limitation** — the availability axis was recomputed under `node-drain`, whose blast radius is placement-dependent — and the corrected design resolves it (§5.8), with a pre-registered confirmatory re-run underway. *Nothing from the original iteration is removed*: its valid findings (the latency dose-response and the conntrack mechanism) are reported in full, and the construction-limited availability results are reported together with their limitation. This iterate-and-disclose trajectory is itself part of the methodological contribution.

1.3 Contributions

The thesis makes three contributions, in priority order:

1. **A pre-registered, provenance-gated measurement methodology (ChaosProbe).** A placement-aware chaos-evaluation framework — a placement solver driving the two knobs, a LitmusChaos experiment runner, cross-layer probers (Prometheus mechanism metrics, route latency with a dependent-vs-control confound split, EndpointSlice availability snapshots), Locust load, and a Neo4j dependency graph from which the cross-node fraction is computed — run under a frozen pre-registration, a `doctor -strict` provenance gate, and freeze-before-analysis, and able to *surface and correct its own construction limitations* (§5.8). The method and its discipline are reusable on any cluster/workload and stand independent of the empirical outcome.

2. **A positively-established, precisely-bounded mechanism.** Placement reproducibly moves a UDP-conntrack reconvergence signature — individually significant in every campaign — and a NodeLocal DNSCache intervention removes it. Under strict, Holm-corrected testing, however, this mechanism does *not* translate into the user-visible placement, availability, and dose-response advantages the hypotheses predicted (no confirmatory hypothesis is supported at its registered bar, §5.6). Bounding exactly how far a real mechanism reaches — and converting "we observed effects" into "here is what survives a strict bar" — is the empirical contribution.

3. **A reproducible campaign protocol with archived artifacts.** Independent single-commit sessions as the unit of analysis, each gated by `doctor -strict`, with every campaign's raw data frozen before its analysis was written and every quoted number traceable to an archived run (Appendix A).

What the candidate did versus what the tooling did. *Authored for this thesis*: the placement solver and the two-knob design; the three-layer measurement design with its dependent-vs-control route confound check; the cross-node-fraction metric and the EndpointSlice trough measurement; the layered scorecard; the pre-registration, the `doctor -strict` provenance gate, the discard-not-patch rule, and the freeze-before-analysis protocol; the churn-vs-node-failure fault-class framing; and every committed analysis script (the per-hypothesis drivers, the family Holm correction, figure generation). *Integrated off-the-shelf*: LitmusChaos (fault execution), Prometheus/Locust (collection and load), Neo4j (graph storage), and the Online Boutique application. The science — what to vary, what to measure, what to believe — is the candidate's; the third-party components execute it.

1.4 Thesis outline

Chapter 2 positions the thesis against four bodies of literature — resilience profiling, production resilience testing, scheduler/placement research, and measurement methodology — and locates it at their empty intersection. Chapter 3 describes ChaosProbe and its provenance-capture design. Chapter 4 defines the three-layer measurement design, the two fault classes, the pre-registered campaign protocol, the statistical methods, and the pinned cluster environment. Chapter 5 reports H1–H5 bar-first with the family Holm capstone and per-claim provenance, an exploratory second-workload external-validity replication (§5.7), and a design-corrected re-analysis that addresses the availability-axis tests' construction limits (§5.8). Chapter 6 interprets the results as a real-but-bounded mechanism and derives the operator-facing implications. Chapter 7 states the threats to validity and the boundary of every claim. Chapter 8 concludes and lays out future work. Appendix A contains the archived-run provenance tables.

Chapter 2

Related work

2.1 Positioning: the quadrant this thesis occupies

The four nearest works, and the axis on which each differs:

Work	What it does	How this thesis differs
MicroRes — Yang et al. (ISSTA 2024), arXiv:2212.12850	Score-based resilience profiling: ranks degradation by whether it disseminates from system to user metrics.	**"They score the decoupling; we measure its mechanism"** under a manipulated variable (placement). MicroRes performs no variance decomposition, ICC, or test-retest reliability analysis of its index — this thesis's scorecard-reliability audit (H5) is that gap. Scope it *against* their reported 0.86–0.90 accuracy / 0.92–0.95 F1: those are for **binary** resilient-vs-non-resilient classification with per-dataset tuned thresholds, a different task from the test-retest reliability of a continuous score across repeated conditions.
Cast — ICSE 2026 SEIP, arXiv:2602.00972	Production resilience testing at Huawei Cloud: traffic record-and-replay + application/RPC-level fault injection (137 potential vulnerabilities, 89 confirmed).	Cast finds *application-layer fault-handling bugs*; this thesis measures *infrastructure-layer placement mechanisms*. Cast has no placement, scheduling, kernel/network-mechanism, or blast-radius content.
Cloud-edge fault-injection study — Liu et al. (2025), arXiv:2507.16109	Largest existing K8s failure-injection dataset (11,965 experiments), architecture-level benchmarking of cloud-edge deployments.	Benchmarks **where the cluster runs, not where pods land**: no churn-vs-node-failure distinction, no conntrack/kube-proxy mechanism, no placement knob. Complementary and orthogonal.
NetMARKS — Wojciechowski et al. (INFOCOM 2021), DOI 10.1109/INFOCOM42981.2021.9488670; **TraDE** (2024), arXiv:2411.05323	Locality as an *optimization objective*: network/traffic-aware schedulers that co-locate using runtime telemetry (NetMARKS: up to 37 % response-time reduction).	They *optimize* locality with runtime metrics; this thesis instead *manipulates* the cross-node dependency-edge fraction as a controlled variable and measures the kernel/network mechanism it moves (the conntrack reconvergence signature) and its dose-response on the east-west tail (H1). The locality concept itself is theirs; NetMARKS reports end-to-end response time, not east-west p95 — directional precedent, not like-for-like.

These four works define the axes of the space this thesis sits in: MicroRes and Cast evaluate resilience without touching placement, while NetMARKS/TraDE and Liu et al. vary deployment topology without chaos-style evaluation of the outcome. The intersection — manipulate placement as a controlled variable, inject faults from distinct classes, measure the mechanism and user layers simultaneously, and audit the reliability of the resilience score current practice would use instead — is empty, and that intersection is where this thesis sits.

Stated as coordinates: on the axis *what is manipulated*, this thesis sits with the schedulers (placement) and against the profilers; on the axis *what is measured*, it sits below the user-visible steady state, at the kernel/network mechanism layer, where none of the four neighbors measure; on the axis *what is trusted*, it treats the evaluation metric itself as an object of study rather than an instrument taken on faith. The rest of this chapter walks the supporting lineages: the scheduler literature that supplies the placement knobs and the predictions (§2.2), the chaos-engineering tradition that supplies the faults and the gap (§2.3), the measurement-methodology literature that supplies the study's argument shape (§2.4), and the tail-latency work that supplies the reporting discipline (§2.5).

2.2 Scheduler & placement lineage

- **Borg** — Verma et al. (EuroSys 2015) and **Borg/Omega/Kubernetes** — Burns et al. (ACM Queue 2016): production schedulers optimize for resource fit, not fault isolation — the gap that motivates studying placement's resilience consequences directly.
- **Medea** — Garefalakis et al. (EuroSys 2018): the canonical "spread-is-best" topology-constraint reference, the source of the placement-dependence prediction H2 tests; valid for the multi-replica availability regime it was written for.
- **Quasar** — Delimitrou & Kozyrakis (ASPLOS 2014) and **Bubble-Up** — Mars et al. (MICRO 2011): the interference/contention model behind the expectation that co-location is costly — written for the contention regime, not the churn regime this thesis's `pod-delete` fault realizes.
- Supporting graph-partition lineage for the cross-node-fraction knob: **DeathStarBench** — Gan et al. (ASPLOS 2019), **Sinan** — Zhang et al. (ASPLOS 2021), and **METIS** — Karypis & Kumar (1998).

This lineage serves the thesis in two roles: it supplies the *placement knobs* and it supplies the *predictions*. On the knob side, placement is manipulated through two orthogonal, continuously-varying controls rather than a set of named strategies (§4.3): the **cross-node fraction** f — the fraction of inter-service dependency edges that cross a node boundary, spanning packing (f=0) to spreading (f=1) — operationalizes the locality axis of the network-aware schedulers [Wojciechowski et al., 2021, TraDE], computed by a lightweight balanced graph partition in the spirit of the microservice-topology literature [Gan et al., 2019, Zhang et al., 2021, Karypis and Kumar, 1998]; and the **replication degree** r with packed vs anti-affine modes operationalizes Medea-style failure-domain separation [Garefalakis et al., 2018]. Sweeping f continuously is what makes the dose-response question (H1) and the static cross-node-fraction predictor testable, rather than contrasting a handful of discrete placements.

On the prediction side, the lineage yields the literature expectations the study tests confirmatorily. The locality optimizers predict that spreading service dependencies across nodes moves the network mechanism — the basis of H2's placement-dependence arm. Medea's failure-domain reasoning predicts that anti-affine replication rescues availability under node failure — the basis of H3. The contention model [Mars et al., 2011, Delimitrou and Kozyrakis, 2014] predicts co-location is costly, but for a regime (resource contention, multi-replica availability) distinct from the single-replica churn this thesis injects; the results (§5.6) find the mechanism these works describe is real and reproducibly moved by placement, while the user-visible placement advantages they motivate do not reach the pre-registered bar in this regime — a scoping result about where the prescriptions transfer, not a refutation of the regimes they were written for.

2.3 Chaos-engineering lineage

- **Principles** — Basiri et al. (IEEE Software 2016); **Chaos Monkey/Simian Army** — Netflix (2011): fault-injection-by-killing, which this thesis generalizes across controlled placements.
- **LitmusChaos**: the injection engine driven here (pod-delete, hogs, node-drain).
- **ChaosEater** — Kikuta et al. (ASE 2025 NIER): LLM-automated chaos — orthogonal; situates the 2025 landscape.
- **Gap evidence** — the ACM CSUR multi-vocal review (arXiv:2412.01416, DOI 10.1145/3777375; ≈90 sources through April 2024) defines chaos outcomes solely against steady-state user-visible indicators and contains no placement, kernel/scheduler, conntrack, or EndpointSlice content; the December 2025 SLR (arXiv:2512.16959) adds a recovery-pattern taxonomy and an evaluation-score checklist, likewise without placement variation. Both are evidence the placement-aware, cross-layer, metric-reliability angle is **a specific gap, not a field-wide void**.
- Adjacent K8s failure analysis: **Mutiny!** — Barletta et al. (DSN 2024), etcd-state corruption (complementary layer); Yang et al. (arXiv:2405.18001), placement-vs-reliability via *modeling* rather than empirical injection.

Chaos engineering begins as a production practice: Netflix's Chaos Monkey terminated instances at random to force teams to build for failure [Netflix Technology Blog, 2011], later codified as a discipline — hypothesize about steady state, inject realistic faults, observe whether the steady state survives [Basiri et al., 2016]. This thesis inherits the central fault of that tradition, the kill, but inverts its key design choice: where Chaos Monkey kills at *random* to exercise unknown weaknesses, here the kill is delivered under *controlled, repeated placements* to measure a specific variable's effect. The injection machinery is delegated to LitmusChaos, a CNCF chaos toolkit whose experiment CRDs (pod-delete, resource hogs, node-drain) the tool drives programmatically (§3.1). At the automation frontier, ChaosEater [Kikuta et al., 2025] uses LLMs to design and run chaos experiments end to end — orthogonal to this thesis's question, but evidence that the field's current emphasis is on automating the loop, not on auditing the metrics the loop optimizes.

The gap this thesis targets is visible in the field's own syntheses. The ACM CSUR multi-vocal review (arXiv:2412.01416, ≈90 sources through April 2024) defines chaos outcomes solely against steady-state user-visible indicators — latency, error rate, throughput, availability — and contains no placement, kernel/scheduler, conntrack, or EndpointSlice content. The December 2025 SLR on resilient microservices (arXiv:2512.16959) adds a recovery-pattern taxonomy and an evaluation-score checklist, again without placement variation, and the JSS systematic review of chaos experiments [Awad et al., 2025] is likewise silent on placement-strategy variation. These surveys show that the placement-aware, cross-layer, metric-reliability angle is a *specific gap* in a well-consolidated literature, not that the literature is deficient wholesale.

The nearest empirical Kubernetes failure studies work at adjacent layers. Mutiny! [Barletta et al., 2024] shows how corruption of cluster state (etcd) propagates to cluster-wide failures — a control-plane layer this thesis does not touch. Liu et al.'s cloud-edge dataset (§2.1) is the scale benchmark for K8s fault injection but varies environment, not placement. Yang et al. [Yang et al., 2024a] address placement-versus-reliability directly but through analytical modeling — predicting failure reduction where this thesis measures behavior under injected faults. Together these works bracket the contribution: the layer (pod placement within one cluster), the method (empirical injection under a manipulated variable), and the measurement design (cross-layer with a reliability audit) are each individually present in the literature's neighborhood, but not combined.

Two further neighbors mark the methodological and forward-looking edges of the space. Hagedoorn et al. [Hagedoorn et al., 2026] empirically study how *architectural topology* — the structure of the application's service graph — affects microservice performance and energy use; they are a methodology peer in varying a structural property and measuring the consequence, but their manipulated variable is the application's design, where ours is its *deployment* onto nodes with the application held fixed. And recent constraint-based pod-packing work [Priority Matters] shows the scheduling community continuing to refine packing objectives — relevant to §8.2's suggestion that a validated static metric like the cross-node fraction could be integrated into such schedulers as a scoring signal, closing the loop from evaluation back to optimization.

2.4 Methodology precedents

- **Maricq et al. (OSDI 2018), "Taming Performance Variability"** — the argument-shape precedent: quantify the noise, then prescribe what repetition can and cannot detect (≈900k data points, 835 servers). Cite with precision: their CONFIRM tool uses nonparametric CI-width stopping, **not** formal hypothesis-test power analysis.
- **Mytkowicz et al. (ASPLOS 2009)** — measurement bias ("Producing Wrong Data Without Doing Anything Wrong!"); the lineage does not start in 2018.
- **Hoefler & Belli (SC 2015)** — report distributions and CIs, not means.

The study's statistical framing follows an established argument shape in systems measurement: before comparing systems, quantify the environment's own variability and derive what repetition can and cannot detect. The precedent is Maricq et al. [2018], who showed across roughly 900,000 data points on 835 supposedly identical servers that hardware-level run-to-run variability is large enough to undermine naive quantitative comparison, and built CONFIRM to recommend repetition counts from historical variability. We cite this with precision: CONFIRM's stopping rule is nonparametric confidence-interval-width stopping, *not* formal hypothesis-test power analysis. This thesis makes the same move — estimate the measurement noise floor (the A/A calibration block), then fix the smallest effect sizes of interest and per-cell sample sizes against it before collecting confirmatory data (§4.3).

The lineage does not start in 2018. Mytkowicz et al. [2009] demonstrated that measurement bias from factors as incidental as link order and environment size can produce wrong conclusions in careful experiments — the canonical warning that an unexamined measurement pipeline is itself a threat to validity, which motivates this thesis's provenance capture (§3.2). Hoefler and Belli [2015] codify the reporting discipline this thesis adopts: report distributions and confidence intervals, not bare means, and treat nonparametric methods as the default when normality is unestablished — the reason Chapter 4's toolkit is built on rank-based tests, bootstrap intervals, and equivalence testing rather than t-tests on means.

2.5 Tail latency

- **The Tail at Scale** — Dean & Barroso (CACM 2013): shared resources → latency variability → service-quality damage; the source of the tail-first reporting discipline this thesis adopts.

Dean and Barroso [2013] established the canonical chain from shared resources to latency variability to user-visible service-quality damage, and the corollary that the *tail* of the latency distribution, not its center, is what users experience in fan-out architectures. The tail-first convention shapes the measurement design: the route probers report p50/p95/p99 per route,

and every latency claim in Chapter 5 is a tail claim (p95), not a mean claim — in particular the east-west p95 that is H1's dose-response outcome.

Finally, what is *not* in the literature. Our searches — general web search, arXiv, Semantic Scholar, and Google Scholar's surfaced results — found no peer-reviewed paper that frames `pod-delete` as a churn fault distinct from contention faults, none that identifies conntrack/EndpointSlice reconvergence as the dominant mechanism under pod-delete on small clusters, and none that manipulates a continuous placement knob under chaos while auditing the resilience score's reliability. We state this as the result of a bounded search rather than proof of absence: ACM DL, IEEE Xplore, and recent USENIX proceedings were not exhaustively swept, and practitioner venues (KubeCon, SRECon, Linux Plumbers) may document parts of the kill-cycle mechanism outside the peer-reviewed record. Within those bounds, the churn-versus-contention mechanism framing appears to be a novel contribution.

Chapter 3

The ChaosProbe framework

ChaosProbe is the instrument that produces every number in this thesis: a Python framework that mutates pod placement as a controlled variable, injects LitmusChaos faults into the target application, probes the outcome at three measurement layers concurrently, and emits structured, provenance-stamped results. This chapter describes its architecture as built (the consolidated technical reference lives in the repository alongside the code) and the provenance-capture design that Chapter 4's campaign protocol depends on. The framework is deliberately *evaluation-shaped* rather than *optimization-shaped*: unlike the network-aware schedulers of §2.2, it does not try to find a good placement — it realizes a chosen placement exactly, disturbs it, and measures what happens at each layer.

Each experiment iteration follows the pipeline in Figure 3.1. The placement mutator first realizes the strategy under test and waits for the deployment to settle; the load generator and the continuous probers then start, the fault is injected and held for the chaos window, and probing continues through a post-chaos phase so recovery is captured. The per-iteration data is aggregated into a per-strategy summary, stamped with the run's provenance manifest, and — for campaign sessions — archived as the citable artifact. The chaos runner, the load generator, and every continuous prober run in parallel during the chaos window; their outputs are joined only after the iteration ends, so no prober's sampling cadence depends on another's.

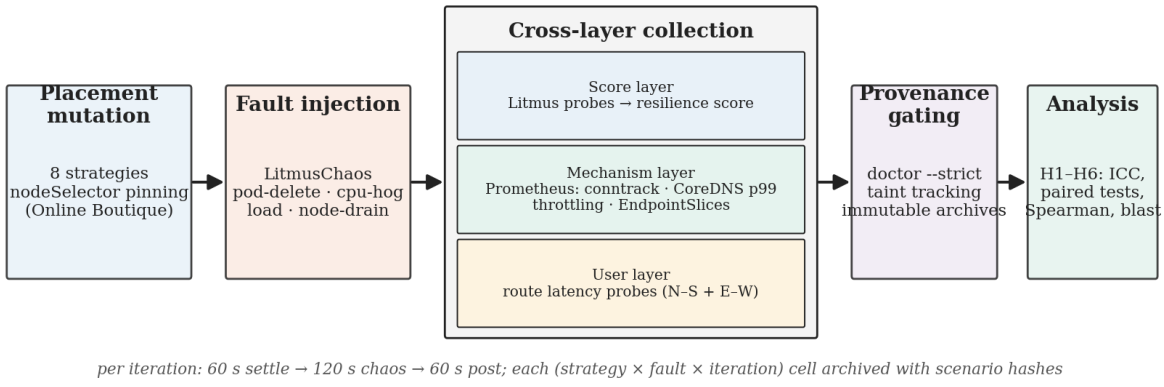

Figure 3.1: **ChaosProbe experiment workflow** — placement mutation → settle → load + chaos injection → post-chaos probing → aggregation → provenance stamping → archive.

3.1 Architecture

The implementation is a single Python package whose top-level modules map one-to-one onto the responsibilities below: `placement/` (strategies and the mutator), `chaos/` (the LitmusChaos runner), `orchestrator/` (phasing, readiness, recovery, diagnostics), `metrics/` and `collector/` (the probers and result collection), `loadgen/` (Locust), `storage/` (Neo4j), `output/` (summary generation, charts, reports), and `commands/` (the CLI surface). The description below follows that layout, so each architectural claim in this chapter is checkable against a named module.

CLI (`chaosprobe`). The command-line interface is the framework's only entry point, organized around the experiment lifecycle: `init` installs cluster infrastructure (LitmusChaos, Prometheus, Neo4j, the in-cluster registry), `run` executes a strategy × fault × iteration matrix, and the analysis commands — `doctor`, `stats`, `power`, `report`, `recommend`, `summarize`, `inspect`, `diff`, `export` — consume the resulting `summary.json` without touching the cluster. A `cluster vagrant` command group manages the Vagrant-provisioned local cluster (§4.5), and archiving a finished run into the citable `dist/` store is performed by the committed `scripts/archive_run.py`. Two properties matter methodologically. First, `run` is **self-healing**: it verifies and reinstalls missing infrastructure before each invocation, so experiments are re-entrant and a campaign session never silently runs against a half-provisioned cluster. Second, the analysis commands are pure functions of the stored output: every statistic in Chapter 5 can be recomputed from an archived `summary.json` without cluster access.

Placement mutator & strategies. Placement is manipulated at the deployment level: the mutator (`placement/mutator.py`) patches each deployment's `nodeSelector` to pin services to nodes, realizing eight strategies defined in `placement/strategy.py` — `baseline` (a no-fault control that leaves placement and injects nothing), `default` (the unmodified Kubernetes scheduler), `colocate` (all services on the target's node), `spread` (services distributed across workers, deterministic per-service pinning), `adversarial` (a worst-fit resource hotspot), `random` (seeded, so reproducible), `best-fit` (bin-packing onto the fewest nodes), and `dependency-aware` (a BFS partition of the service dependency graph). These eight are the framework's general-purpose placement primitives; the pre-registered study does not select among them directly but drives placement through the two continuous knobs — the cross-node fraction f and the replication degree r — that §2.2 anchors to the scheduler literature, with the solver realizing the per-service pinning that hits each target f. Because a `nodeSelector` is a request to the scheduler, not a guarantee of the realized state, the mutator verifies the outcome: each iteration records per-service `placementMatchRates` — the fraction of pods that actually landed where the strategy intended — so mismatched iterations are visible to the analysis rather than silently polluting it (§7.1). Manipulating f at one replica per service is a scoping decision with consequences the thesis returns to repeatedly: it cleanly varies *between-service* topology, while *within-service* replication is exercised only by the r knob in H3's node-drain campaign — multi-replica anti-affinity beyond that regime is out of scope (§7.2, §8.2).

LitmusChaos runner. Fault injection is delegated to LitmusChaos: the runner (`chaos/runner.py`) saves, triggers, and polls ChaosEngine experiments (`pod-delete`, CPU/memory hogs, `node-drain`) through the ChaosCenter GraphQL API, wrapping each in the settle → chaos → post phasing that gives the probers stable pre- and post-fault baselines. The aggregate resilience score of §4.1 is computed from these experiments' probe verdicts — including command probes whose images `run` builds and pushes to an in-cluster registry — which is why the score's failure modes track the probes' own dependencies on cluster health (§6.1). The runner carries two pieces of hardening that campaign-scale operation forced. Injection attempts retry with best-attempt quality handling: a transient Litmus failure retries, and if no attempt is fully clean the best attempt is kept *and marked*, so the data-quality gate (`doctor`) can taint or exclude the iteration instead of the run aborting or the defect passing unnoticed. And destructive faults have cleanup guards — after `node-drain`, the runner uncordons any node left cordoned by a failed experiment teardown, because a leaked cordon would silently change the cluster available to every subsequent

iteration.

Orchestration, readiness gates, and recovery measurement. Between the mutator and the runner sits the orchestrator, which owns the property the statistics of Chapter 4 quietly depend on: that every iteration of every strategy is measured under the same protocol. A pre-flight module validates the scenario and cluster state before each iteration; readiness gates hold the iteration until the target pod is running, the application answers end-to-end, and a warmup period has passed — so the pre-chaos baseline is a settled system, not a deployment still converging from the previous placement mutation. Timing arithmetic derives the probe windows from the declared chaos duration, so settle, chaos, and post-chaos phases are identically proportioned across strategies and sessions. A recovery watcher follows the target deployment's pod lifecycle through the kill cycle, recording deletion, scheduled, and ready timestamps — the source of the recovery times used in the availability analysis. When a Litmus probe returns an `Unknown` verdict, a diagnostics module captures the cluster state at that moment, which is how the study can assert *why* the Litmus score is unusable under node-drain — the reason H3's availability is measured from EndpointSlice troughs (§5.3) — rather than merely observing that it is.

Probers (cross-layer). The measurement design of §4.1 is realized by a set of continuous probers that run concurrently through every iteration. The *Prometheus mechanism prober* samples kernel- and infrastructure-layer signals via PromQL: `conntrack_entries_per_node` (the H2 signal), CoreDNS request-duration tails, TCP retransmits, and CPU throttling; kube-proxy's own network-programming-latency metric is queried but recorded as uncollected on this cluster (§7.1) — the framework's `metricAvailability` map distinguishes "not collected" from "collected zero" precisely so such gaps cannot manufacture fake values. The *route latency prober* measures the user layer: per-route p50/p95/p99 and error rate against the application's HTTP routes, with routes classified as fault-**dependent** (they touch the chaos target) or **control** (they do not) — the confound-control split that H3 and H4 rest on. *Redis, disk, and resource probers* track application state, I/O, and node/container resource trajectories. Finally, an *EndpointSlice snapshotter* records per-service ready-endpoint counts every 15 s through the chaos window; its outage troughs are the basis of the trough-depth blast-radius metric H3 uses, which deliberately bypasses both the score and the route layer (under a node-drain, every Litmus probe returns `Unknown` and the score is unusable — §5.3).

Load generation. User traffic is generated by Locust profiles against Online Boutique's user-facing routes: a steady profile (50 users) provides background traffic for churn experiments, and a 200-user spike profile provides elevated user load during the chaos window. The load generator runs in parallel with the probers throughout the chaos window, so the route prober measures latency under the same traffic the fault perturbs.

Storage. Each run emits structured per-iteration JSON aggregated into a per-strategy `summary.json` (output schema 2.0.0) — the single artifact all analysis consumes — and writes a **Neo4j graph** of services, dependency edges, and per-iteration pod→node placements. For most of the study the graph is storage; the cross-node fraction knob makes it **analytically load-bearing**: the cross-node call fraction f — the placement knob the study manipulates — is computed by joining the graph's inter-service edges with the recorded `podPlacements`, so each placement's realized f is known *before any chaos is injected* and depends on the graph being a faithful model of the application's call structure. The `summary.json` itself is organized for exactly the analyses Chapter 5 performs: per strategy it carries the per-iteration scores, probe verdicts, recovery timings, placement match rates, the per-route latency/error aggregates split by dependent-vs-control classification, the mechanism metric time-series summaries with their `metricAvailability` flags, and the `routeViewAggregate` from which the east-west inter-service tails of H1 and H4 are read. Nothing downstream re-queries the cluster; the file is the experiment's complete, self-describing record.

Analysis commands. Downstream analysis is split between the CLI and committed cam-

paign scripts. `doctor` is the data-quality gate: it checks per-iteration completeness, probe verdicts, and placement match, and in `-strict` mode additionally refuses runs with incomplete provenance (§3.2). `stats` computes pairwise strategy statistics; `report` renders HTML/Markdown reports; `recommend` summarizes the per-strategy comparison. The campaign-level numbers in Chapter 5 come from dedicated scripts — `scripts/c1_h1_trend.py` (H1, Page's L dose-response), `c3_h2_dns.py` (H2, placement + DNS), `c2_h3_anova.py` (H3, ART ANOVA replication rescue), the frontier driver (H4), `scorecard.py` (H5 layered-scorecard ICC), `holm_family.py` (the family Holm correction), and `campaign_status.py` (session bookkeeping) — each committed in the repository, so every quoted figure names the code path that produced it.

Three design principles cut across these components and are worth stating once. First, **measurement is separated from judgment**: the probers record what happened — including taints, partial data, and `Unknown` verdicts — and the decision about whether an iteration's data is usable belongs to `doctor`, applied after the fact under explicit, versioned rules. Nothing in the collection path drops inconvenient data. Second, **absence is recorded, not interpolated**: the `metricAvailability` map exists because the most dangerous failure mode of a metrics pipeline is a missing query that reads as zero; a metric ChaosProbe could not collect is stored as uncollected, and the analysis scripts treat it as such. The kube-proxy network-programming-latency metric is the live example — queried, never scraped on this cluster, and therefore cited in this thesis only as conceptual anchoring for the mechanism, never as data (§7.1). Third, **every number is regenerable**: the chain from CLI flags to scenario YAML to `summary.json` to analysis script is deterministic given the archived inputs, which is what allows Appendix A to function as a claims→runs mapping rather than a bibliography of irreproducible measurements.

The framework's limitations are part of its honest description. Placement is mutated at deployment level with `nodeSelector`, which is what makes the realized placement verifiable: the f knob varies between-service topology at one replica per service, and the r knob reaches r=3 packed/anti-affine through round-robin per-replica pinning in H3's node-drain campaign (§5.3). Multi-replica anti-affinity beyond that node-drain regime is out of reach of the current mutator, a boundary that shapes the future-work design in §8.2. The probers sample at fixed cadences (EndpointSlice snapshots every 15 s), so sub-cadence transients are invisible; the blast-radius troughs (H3) are well above this resolution, but finer-grained availability dynamics would need a faster snapshotter. And the framework measures one cluster at a time: nothing in the design prevents running the same campaign on different infrastructure, but the cross-environment comparison itself is future work, not a shipped feature.

3.2 Provenance capture

Every run is stamped with a manifest (`artifact-manifest.json`) carrying:

- **scenario SHA-256 hashes** (the exact fault YAML injected),
- **batch/day IDs** and run ID,
- **kube-proxy mode and conntrack settings** (mode, `maxPerCore`, `min`, TCP timeouts) — required because the H2 mechanism is version- and mode-sensitive,
- **git commit** (+ dirty flag) of the framework code,
- **environment fingerprint**: Kubernetes server version, container runtime, node OS, CNI hint, namespace, strategy/fault matrix, iteration count.

`doctor -strict` refuses a run whose provenance is incomplete or whose scenario hashes drift; `scripts/archive_run.py` banks the run under `chaosprobe/dist/` as the citable artifact (Appendix A).

Provenance is a first-class feature of the framework, not packaging convenience, for three reasons. First, the claims demand it: the H2 mechanism is sensitive to the Kubernetes version, the kube-proxy mode, and the kernel's conntrack configuration (§6.2 — upstream conntrack handling changed materially across v1.31–v1.32), so a conntrack-flush number quoted without its environment fingerprint is not interpretable, let alone reproducible. The manifest pins exactly the parameters on which the mechanism's scope depends.

Second, weak provenance has an empirical cost. An early, weakly-provenanced pilot run — with untracked working-tree changes and incomplete metadata — produced a striking user-layer number that did not survive: when the measurement was repeated under `doctor`-gated, strict-clean runs, the user-layer effect collapsed and only the mechanism-layer effect persisted. Had the un-gated pilot been quoted, a headline result would have been an artifact of an unreproducible run state. The lesson is institutionalized as the campaign rule that no number is quoted from a run failing `doctor -strict`, and as the manifest's dirty flag, which makes an uncommitted-code run permanently visible as such.

Third, the external advisory review of this work flagged missing raw artifacts as a blocker: findings quoted in prose with no bankable evidence trail. The archive pipeline answers that structurally rather than procedurally — `scripts/archive_run.py` packages each run's `summary.json`, per-strategy data, charts, and manifest, with per-file SHA-256 hashes, under `chaosprobe/dist/`, and Appendix A maps every claim in Chapter 5 to the archives that carry it. The provenance chain is thus verifiable end to end: from a number in this document, to a named archive, to hashed data files, to the exact scenario YAML and code commit that produced them.

Chapter 4

Methodology

This chapter defines how the research question of §1.2 is turned into measurements: the three-layer measurement design (§4.1), the two fault classes (§4.2), the *pre-registered* campaign protocol that is itself a contribution (§4.3), the statistical methods and multiplicity control (§4.4), and the pinned cluster environment every claim is scoped to (§4.5). The defining methodological choice is that the confirmatory hypotheses, their smallest effect sizes of interest (SESOIs), the per-cell sample sizes, the outcome operationalizations, and the analysis code were all *frozen before any confirmatory data was collected*; the design that follows is reported as registered, with every later deviation logged.

4.1 Three-layer measurement design

Each experiment is measured at three layers that need not agree — the unit of explanation is *which layer moves*:

1. **Layered scorecard** — a resilience scorecard split into sub-scores (availability, mechanism-reconvergence, user-tail), replacing the single industry-style aggregate number whose reliability H5 audits against a naive single-aggregate baseline.
2. **Mechanism metrics** — kernel/network signals: per-node conntrack entries (the UDP-conntrack reconvergence signature that recurs throughout the results), CoreDNS tail latency, TCP retransmits, and east-west inter-service route tails.
3. **User-visible routes** — per-route tail latency and error rate, with a built-in **dependent-vs-control confound check**: *dependent* routes traverse the chaos target, *control* routes do not, so a genuine fault-specific effect must show on dependent routes *beyond* the control route's run-level drift.

The confound-controlled user layer is the difference between "correlates with the run" and "caused by the fault path." On a small virtualized cluster entire runs are slower or faster than each other for reasons unrelated to the fault (host load, cache state, background reconciliation), so any two metrics measured in the same slow run will correlate. The control routes break this degeneracy: riding the same run but not traversing the killed service, they carry the run-level drift and nothing else, so a genuine mechanism→user propagation must show an association on the dependent routes both significant and in clear excess of the control routes'. A within-run rank correlation (computed inside each run, where run-level drift is constant by construction) confirms the read. Each layer is read by a different instrument that fails independently: the scorecard from LitmusChaos probe verdicts, the mechanism layer from PromQL samples of kernel/infrastructure metrics, and the user layer from a **host-side** route prober's active HTTP measurements under Locust traffic — host-side by construction so the prober occupies no cluster

node the placement solver controls and contributes no in-cluster pod to the per-node conntrack aggregation.

4.2 Fault classes

Two fault classes are used, each matched to the layer it exercises:

- **Churn (`pod-delete`).** Repeated deletion of a target service's pod drives the kernel/network reconvergence the mechanism layer measures. By construction it removes a single pod at a time, so it produces a strong, reproducible conntrack signature but *no sustained endpoint outage* — which is why the availability face is degenerate under this fault (§5.4) and the availability scorecard layer has little reproducible signal to estimate under it (§5.5). Used by H1, H2, H4, H5.
- **Node failure (`node-drain`).** Draining a worker node removes whole pods' worth of endpoints at once, producing the sustained availability trough the replication-rescue hypothesis (H3) needs. It is run with host-side load on the user route; it carries no pre-chaos east-west latency face, which is why its cells cannot be placed on the latency $\leftrightarrow$ availability frontier (§5.4).

4.3 Pre-registered campaign design

Two knobs, not eight strategies. The design replaces an ad-hoc set of named placement strategies with two orthogonal, continuously-varying knobs the placement engine targets directly: the **cross-node fraction** f (the fraction of inter-service dependency edges that cross a node boundary, swept over $\{0, 0.25, 0.5, 0.75, 1.0\}$) and the **replication degree** $r \in \{1, 3\}$ with packed vs anti-affine modes. Two **interventional arms** are added whose predicted effects follow from the conntrack mechanism: a NodeLocal DNSCache toggle (H2's DNS arm) and the kube-proxy mode (H6, not attempted — §5.9).

Pre-registration and provenance discipline. The hypotheses, SESOIs, per-cell n, outcome operationalizations, and analysis code were frozen at a tagged commit *before* any confirmatory data was collected; each campaign's raw data was likewise archived and hash-stamped *before* its analysis was written. Every session records a provenance fingerprint and must pass a `doctor -strict` gate (clean working tree, complete metadata) to be admissible; the registered exclusion rule discards any rejected or fully-tainted session. This protocol — pre-registration, freeze-before-analysis, and a strict provenance gate — is one of the thesis's contributions, not just its hygiene.

The confirmatory family and the descriptive deliverables. Four hypotheses form the confirmatory family corrected together (§4.4): H1 (dose-response of the east-west tail, complete-block design over the five f-levels), H2 (placement-dependence + the DNS intervention), H3 (replication rescue under node-drain), and H5 (layered-scorecard reliability). H4 is a *descriptive* latency$\leftrightarrow$availability frontier (a registered figure and reporting protocol, no p-value), and H6 is an exploratory secondary that was not attempted. The campaigns map onto these: **C1** (8 complete-block `pod-delete` sessions) supplies H1 and H5; **C2** (24 `node-drain` sessions, three cells) supplies H3; **C3** (14 `pod-delete` sessions, cache-off/on) supplies H2 and closes the family.

Feasibility and calibration gates. Three pre-data gates de-risked the design. **M1a** was a cheap solver-feasibility spike (could the engine hit the designed f-levels within ± 0.05?) run on the existing small cluster. **M1b** re-verified the solver *and* the capacity arithmetic on the actual run cluster; it passed (all five levels within ± 0.05, 3/3 consecutive attempts), so the designed-level dose design and Page's test stand rather than the pre-registered continuous-fraction fallback. **M2** was an A/A calibration block (same-placement pairs, null data) that

estimated the measurement noise floor; the SESOIs and dominance margins were then fixed at or above that floor under the registered "no smaller than the A/A noise band" rule — the H1 SESOI exceeds it, while the H3 rescue/TOST and H4 dominance margins are set equal to the band. The H1 SESOI ($\geq 15\,\%$), for example, was set between the measured A/A noise band ($\approx 11.2\,\%$ of the pre-chaos level) and the larger separation seen in an earlier internal pilot, so that an effect at the pilot magnitude would clear it with margin while anything inside instrument noise could not.

4.4 Statistics

Per-hypothesis primary tests. H1 uses **Page's** L trend test (tie-corrected) over the ordered f-levels, with session-condition medians as the unit — the design is replicated complete ordered blocks, which is exactly what Page's test assumes. H2 is a two-part both-must-pass conjunction of paired one-sided **Wilcoxon signed-rank** tests on the during-churn UDP-conntrack drop (placement-dependence and the $\geq 50\,\%$ DNS-cache shrinkage). H3 uses **ART ANOVA** (aligned-rank transform) for the $r \times$ mode interaction, with the registered rescue margin and a packed$\approx$ r=1 **TOST equivalence** control on each co-primary. H5 reports condition-level **intraclass correlations** (ICC, cluster-bootstrap CIs) for the scorecard sub-scores against the naive aggregate baseline, with an absolute reliability bar of ICC ≥ 0.5.

Support requires significance *and* effect size. Every confirmatory verdict is gated on two things: Holm-corrected statistical significance, *and* clearing the registered effect-size or reliability bar (the SESOI). A statistically detectable effect below its SESOI is reported as below the bar, not as support — the discipline that distinguishes a real but negligible trend (H1) from a meaningful one.

Multiplicity — the family Holm correction. The four confirmatory primaries are corrected together by the **Holm** step-down procedure at $\alpha = 0.05$, computed once all campaigns land (§5.6), each member's family-input p read verbatim from its own analysis driver. H4's descriptive frontier and the exploratory secondaries are outside the family and carry no confirmatory weight. TOST equivalence testing [Schuirmann, 1987, Lakens, 2017] is used where a claim is that an effect is *demonstrably small* rather than merely undetected; the distribution-first reporting (medians, tails, cluster-bootstrap intervals rather than bare means) follows the systems-measurement guidance of Hoefler & Belli (SC 2015), and the quantify-then-prescribe argument shape follows Maricq et al. (OSDI 2018).

4.5 Cluster environment

- **Cluster**: kubeadm cluster of 1 control plane + **8 worker nodes (2 vCPU / 4 GiB each)** on Vagrant-managed KVM/libvirt VMs on a single host — the capacity-feasible cluster on which the M1b solver-and-capacity gate was re-verified before collection.
- **Kubernetes v1.28.6** (pinned — kube-proxy/conntrack behaviour changed materially in v1.31–v1.32, see §6.2), containerd 1.7.11, Ubuntu 22.04 LTS nodes.
- **kube-proxy mode:** `ipvs`, conntrack settings archived per run — the reason H6 (an iptables-mode comparison) was de-scoped.
- **Workload**: Online Boutique (Google Cloud microservices-demo), 10 polyglot gRPC microservices + Redis, single replica per service in the baseline regime, host-side Locust load generator.

All VMs run on a single physical host under KVM/libvirt, making host-level interference a shared, time-varying influence on every measurement. This is treated as a property of the

environment *absorbed by design* — the session-level blocking of the campaign protocol (§4.3) and the dependent-vs-control route split (§4.1) are precisely the controls for it. The environment values are stamped into every archived run's manifest so each mechanism-level claim carries its exact scope (§3.2); what does and does not port out of this pinned environment is stated in §7.2.

Chapter 5

Results

This chapter reports the pre-registered confirmatory study. The four confirmatory hypotheses (H1, H2, H3, H5) are corrected together by Holm across the campaigns; H4 is a descriptive frontier (not a falsifiable member); H6 was not attempted (§5.9). Each section states its *registered bar first*, then the outcome against that bar — a hypothesis counts as supported only if it is both Holm-significant *and* clears its registered effect-size / reliability bar. The confirmatory family is corrected and adjudicated together in §5.6.

5.1 H1 — Dose-response of the east-west tail

Registered bar. H1 predicts that median east-west (inter-service) p95 latency increases *monotonically* in the achieved cross-node fraction f across the designed levels $\{0, 0.25, 0.5, 0.75, 1.0\}$. Primary test: Page's L trend test (tie-corrected), session-condition medians of the pre-chaos east-west p95 as the unit. The smallest effect size of interest (SESOI), frozen pre-registration, is a $\geq 15\,\%$ **increase** from $f = 0$ to $f = 1$; a statistically detectable but sub-SESOI trend is reported as below the bar, not as support.

Campaign. Eight complete-block sessions (online-boutique, $r = 1$, `pod-delete` churn), each visiting all five f-levels in randomized order; $5 \times 3 = 15$ iterations per session, all `doctor -strict` clean, all f-targets hit exactly.

Result — trend detected, but below the SESOI.

Page's L	$L = 410$, $z = 3.54$
p (one-sided, uncorrected)	0.0002
per-level median p95 (ms)	35.74 / 38.60 / 41.40 / 39.55 / 40.51
f=0 → f=1 effect	+13.35 %
15 % SESOI	**not met**

A monotone-increasing trend is detected ($p = 0.0002$; Holm-adjusted in §5.6), but the total effect (+13.35 %) sits below the registered 15 % SESOI. The medians rise then plateau/dip slightly at f=0.75 (39.55 ms) — neither a strict monotone climb nor the clean two-regime step that would have triggered the distinct registered "threshold, not dose-response" outcome. **H1 is therefore not supported:** the dose-response is real and statistically significant but practically negligible at the registered bar (Figure 5.1).

A disclosed deviation governs this analysis. H1 (and H5, §5.5) is computed with the registered D3 pre-window UDP-slope taint *withdrawn* from the C1 latency analyses — the study's only *outcome-aware* (non-blind) deviation, logged as D-2026-06-14-02 and decided after observing that the registered taint, applied to C1, flags *every* iteration at f=0.25 and f=0.5: zero complete blocks, so under the gate exactly as registered Page's L is uncomputable and **H1 is unrunnable**. The withdrawal rests on protocol-independent grounds: the slope band

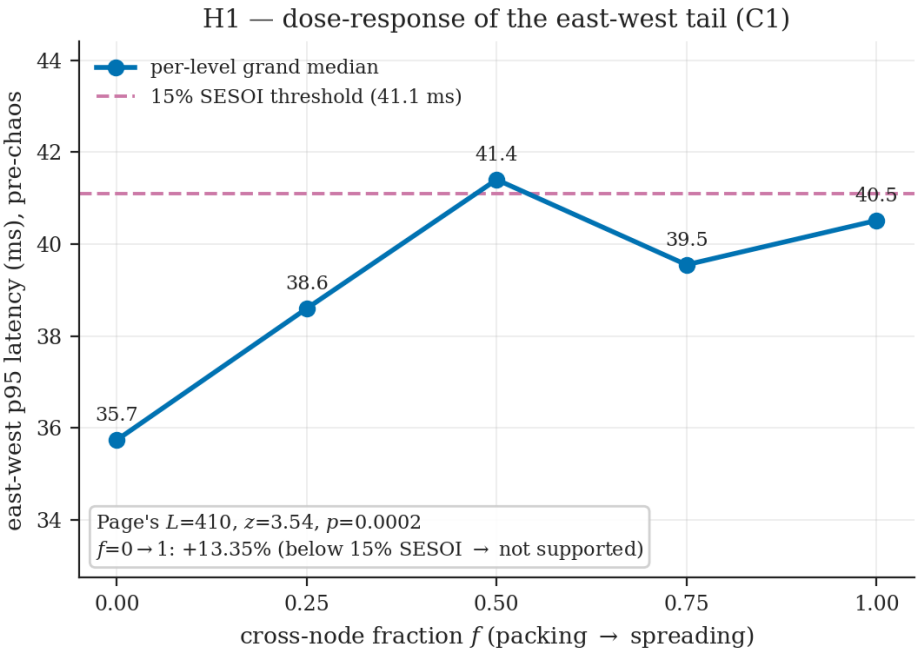

Figure 5.1: **H1 — dose-response of the east-west tail** (C1, r=1, `pod-delete`). Per-level grand-median east-west p95 rises then plateaus/dips at f=0.75; the f=0 → 1 change is +13.35 %, below the registered 15 % SESOI (dashed), so H1 is not supported despite a Holm-significant Page's L trend.

was calibrated on a low-churn affinity-only block and gates the DNS/UDP conntrack pool — a *different* protocol from the TCP/gRPC east-west latency outcome, for which the tainted levels are in fact the *cleanest* baselines — and it reproduces exactly the placement-coupled pre-window transient the M2 preparation had pre-flagged. The result is reported taint-off; the registered taint-on run is available as a sensitivity check but, for H1, yields *no* result rather than a different one. This asymmetry matters: H5's FAIL verdict is robust to the choice (it fails either way, §5.5), whereas H1 has a verdict only with the taint off. Being non-blind, this is the one deviation a reader may weigh differently; it is recorded in full in the deviations log.

5.2 H2 — Placement-dependence and the DNS intervention

Registered bar. H2 is a two-part, both-must-pass conjunction over the during-churn UDP-conntrack drop (`udp_conntrack_drop_entries`): **(a) placement-dependence** — spread (f=1) shows a *larger* drop than packed (f=0), paired; and **(b) DNS mechanism** — enabling the NodeLocal DNSCache shrinks the spread placement's drop by ≥ 50 %. Both arms must pass for H2 to hold.

Campaign. Fourteen sessions (7 cache-off, 7 cache-on), r = 1, placements f=0 (packed) / f=1 (spread), `pod-delete` churn; all 14 accepted, untainted, strict-clean.

Result — the DNS arm passes; the placement arm reverses.

arm	result	one-sided p
(a) placement-dependence (cache-off)	**packed > spread in 7/7 pairs** (packed median 11153.4, spread 8929.1) — direction **reversed**	0.98875
(b) DNS mechanism (within-spread)	**78.0 % median** shrinkage (all 7 pairs 61–85 %) — **passes**	0.01125

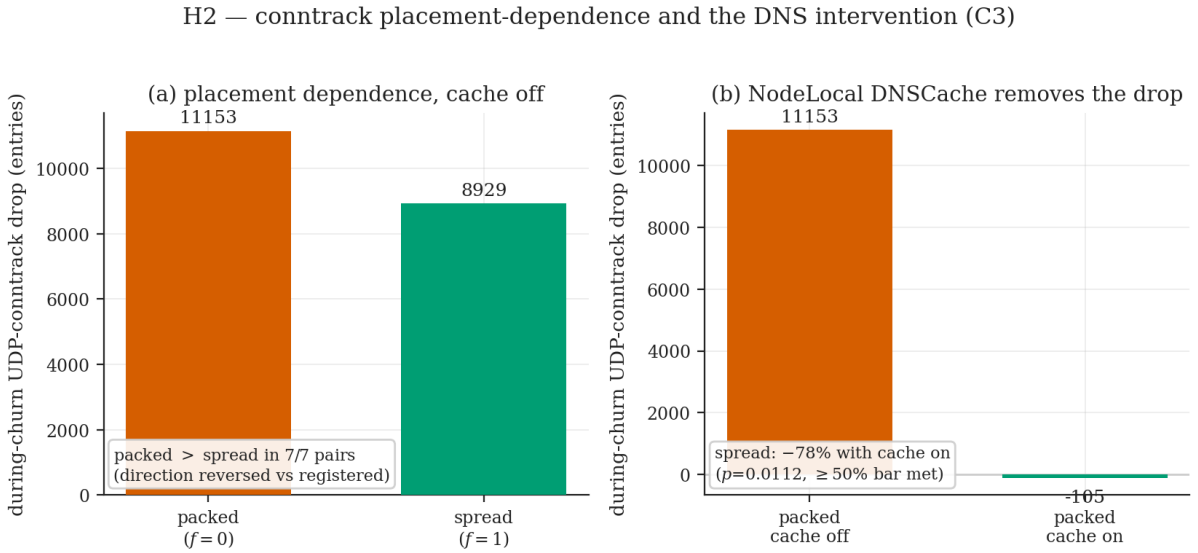

Figure 5.2: **H2 — conntrack placement-dependence and the DNS intervention** (C3). (a) The cache-off during-churn UDP-conntrack drop is *larger* under packing than spreading — reversed against the registered prediction, packed > spread in all 7 pairs. (b) Enabling the NodeLocal DNSCache collapses the packed drop to ≈ 0 and shrinks the spread drop by a 78 % median, removing it under both placements: arm (b) passes, but the reversed direction (a) fails the conjunction.

Conjunction = False; H2 is not supported. Arm (b) is strongly confirmed — the DNS-cache intervention removes the conntrack drop. Arm (a) fails on *direction*, and the reversal is robust and monotone: packed exceeds spread in every one of the 7 cache-off pairs (two-sided Wilcoxon at the $n = 7$ floor, $p = 0.0225$). Co-locating a service's replicas on one node *concentrates* the per-node conntrack churn when `pod-delete` hits that node, producing a *larger* during-churn UDP drop than spread placement, which distributes the same replicas across nodes. Placement *does* matter for the conntrack drop — significantly, but in the opposite direction to the registered prediction. A registered secondary prediction is *not* borne out, and in failing it sharpens the mechanism: the secondary held that the packed (f=0) arm, which has no cross-node dependency edges, would show only a near-floor cache effect *if* the drop were specifically cross-node DNS. Instead the packed cache-off drop is large (median 11153.4) and *also* collapses to ≈ 0 with the cache on (median −105.2; $p = 0.01125$). The intervention thus removes the drop under *both* placements (Figure 5.2), so the floodable pool is the *general* per-node DNS/UDP conntrack churn rather than specifically cross-node DNS resolution — consistent with the reversed arm-(a) direction.

5.3 H3 — Replication rescue under node-drain

Registered bar. H3 predicts an *interaction*: replication (r=3) rescues availability under node-drain only when replicas do not share the failure domain (anti-affine r=3 ≪ r=1; packed r=3 ≈ r=1). Two co-primary outcomes — trough depth and user-route error rate — combined both-must-pass; each requires (i) a significant ART-ANOVA $r \times$ mode interaction, (ii) the anti-affine rescue margin met, and (iii) the packed≈ r=1 equivalence (TOST) control.

Campaign. 24 node-drain sessions (3 cells ×8): r1-packed, r3-packed, r3-anti-affine, on the capacity-feasible round-robin packed instrument; all 24 accepted and untainted.

Result — the error face rescues; the depth face cannot adjudicate.

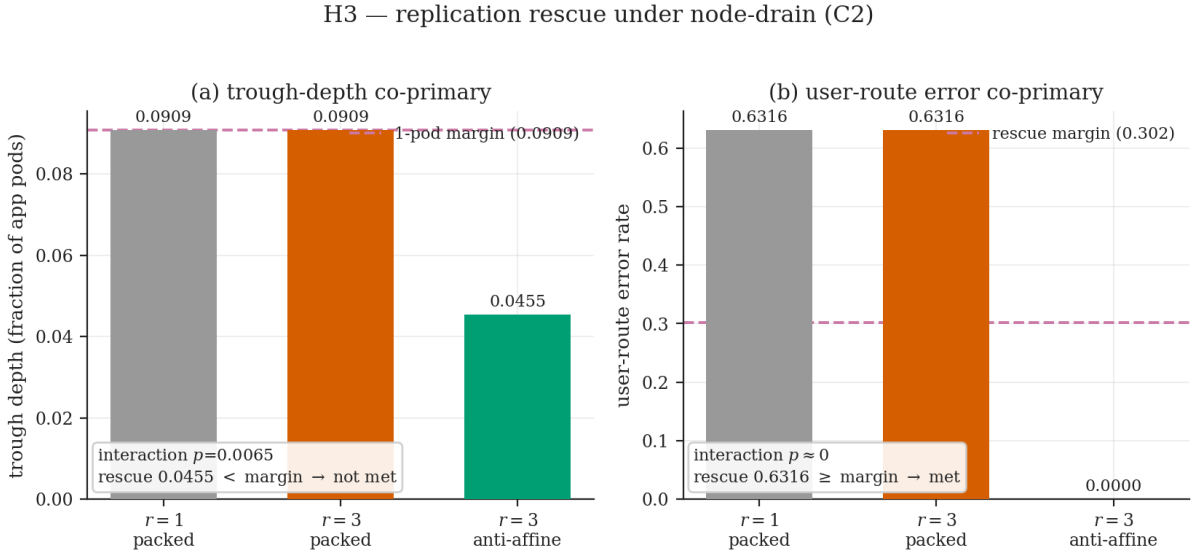

Figure 5.3: **H3 — replication rescue under node-drain** (C2), across the three cells. (a) Trough depth: the anti-affine rescue (0.0455) falls below the registered one-pod margin (0.0909, dashed), which the r=1 and r=3-packed cells sit on by construction — the depth face cannot adjudicate. (b) User-route error: anti-affine r=3 fully rescues (0.632 → 0), clearing the 0.302 margin. The conjunction fails only on the depth co-primary it could not test.

co-primary	r1	r3-pk	r3-anti	verdict vs bar
trough depth (frac)	0.0909	0.0909	0.0455	interaction sig (p=0.0065); rescue 0.0455 < 0.0909 margin → **not met**
user-route error	0.6316	0.6316	0.0000	interaction $p \approx 0$; rescue 0.6316 ≥ 0.302 → **met**; TOST within band

Conjunction = False; H3 is not supported — but the single failing criterion is one the depth co-primary *could not adjudicate by construction*. Both packing controls pass (packed r=3 ≈ r=1 exactly on both faces), so the instrument behaves as designed. On the **user-route error face**, anti-affine r=3 fully rescues the error rate (0.632 → 0.0; interaction $p \approx 0$; rescue ≫ the 0.302 margin) — strong, significant directional support. On the **trough-depth face**, the interaction is significant (p = 0.0065) but the anti-affine rescue (0.0455) falls below the 0.0909 margin; under round-robin spread, draining one node costs r=1 only ≈ 1 pod, so the r=1 depth (0.0909) equals the registered one-pod margin — the metric's dynamic range coincides with the bar it must beat, leaving no room for a rescue to register. The depth co-primary is reported exactly as registered (not retuned); the substantive result is the error face, where replication rescues user-visible availability when replicas are spread (Figure 5.3). A design-corrected re-analysis (§5.8) shows the rescue effect is real once this construction artifact is removed: anti-affine halves the trough depth and eliminates the user-route error.

5.4 H4 — The placement frontier (descriptive)

Protocol (registered, descriptive — no p-value). For each designed placement, the *latency face* (pre-chaos east-west p95 tail) is plotted against the *availability face* (during-chaos trough depth in pods + user-route error rate), and the non-dominated set is reported under frozen

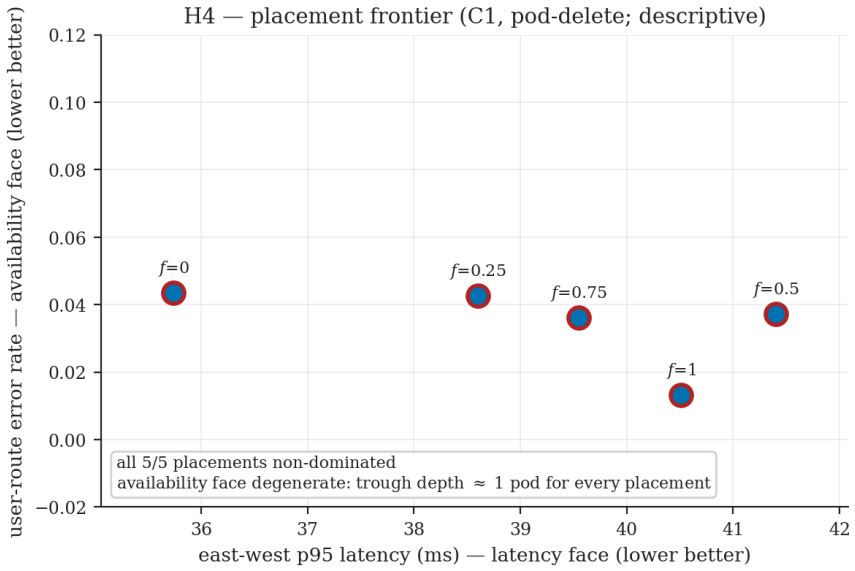

Figure 5.4: **H4 — descriptive placement frontier** (C1, `pod-delete`). All five placements are non-dominated (red rings): the latency face varies (east-west p95 35.7–41.4 ms) but the availability face is degenerate — trough depth is ≈ 1 pod and the error rate uniformly low — so no placement dominates on all faces.

margins ($\delta_{\text{latency}} = 4.4$ ms, $\delta_{\text{depth}} = 1.0$ pod, $\delta_{\text{error}} = 0.302$; dominance requires beating on the latency face *and* both availability DVs).

placement (r=1, pod-delete)	EW p95 [ms]	depth [pods]	user err	non-dom.?
f=0 (packed)	35.74	1.0	0.04	yes
f=0.25	38.60	1.0	0.04	yes
f=0.5	41.40	1.0	0.04	yes
f=0.75	39.55	1.0	0.04	yes
f=1 (spread)	40.51	1.0	0.01	yes

Non-dominated set: all 5/5 placements. The frontier is degenerate on the availability face: `pod-delete` removes a single pod by construction, so trough depth is ≈ 1 pod for every placement and the error rate is uniformly low — neither availability DV separates placements by its margin. The latency face *does* vary (packed 35.7 ms vs mid 41.4 ms, $> \delta_{\text{latency}}$, non-overlapping CIs — the H1 signal), but a single-face advantage cannot establish margin-dominance under the all-DV rule, so the protocol correctly crowns no winner. The two-face latency↔availability trade-off the frontier was designed to reveal is **not realizable from the collected data**: the fault that varies the availability face (node-drain) has no latency face, and the fault with a latency face (pod-delete) produces no availability-face variation (Figure 5.4). A design-corrected follow-up (§5.8) re-draws this frontier under a node-drain dose-response, where the availability face *does* vary (1.0 → 0.36 across placements) and a real trade-off appears.

5.5 H5 — Layered scorecard reliability

Registered bar. H5 evaluates a layered resilience scorecard against a **naive single-aggregate baseline** via condition-level test-retest reliability (ICC), requiring a conjunction of the **availability** and **mechanism** sub-scores each clearing an absolute bar of ICC ≥ 0.5.

Result — mechanism reliable, availability not.

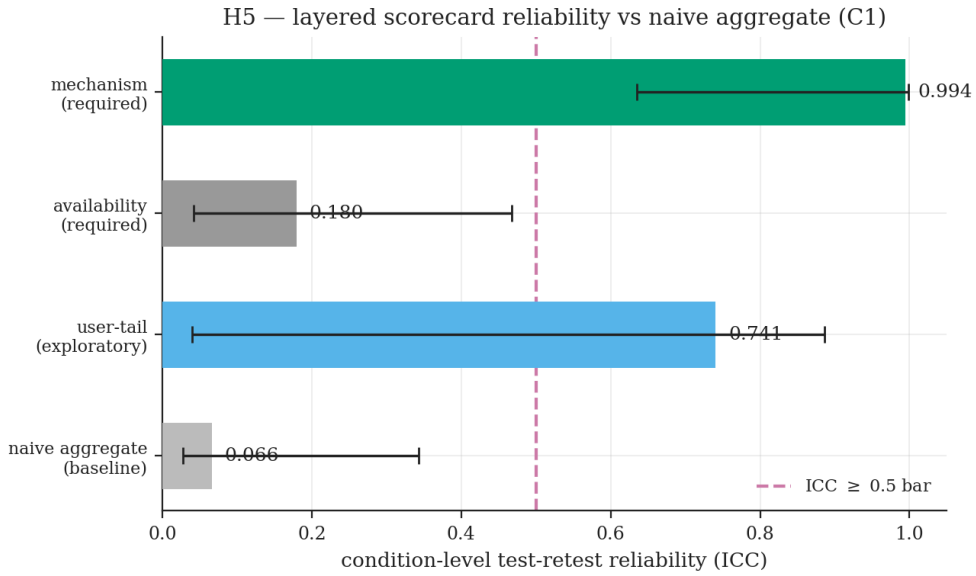

Figure 5.5: **H5 — layered-scorecard test-retest reliability** (C1). The mechanism sub-score (ICC 0.994) clears the registered ICC ≥ 0.5 bar (dashed); the required availability sub-score does not (0.180), so H5 is not supported. Both far exceed the naive single-aggregate baseline (0.066). Bars show 95 % bootstrap CIs.

sub-score	ICC	95 % CI	≥ 0.5?
availability (required)	**0.180**	[0.042, 0.468]	**no**
mechanism (required)	**0.994**	[0.635, 0.999]	yes
naive aggregate baseline	0.066	[0.027, 0.343]	—

The scorecard's third layer, the user-tail sub-score (ICC 0.741, Figure 5.5), is reported for completeness but is not part of the registered required conjunction, which is over the availability and mechanism sub-scores only.

Required conjunction: FAIL; H5 is not supported. This is a sensible registered falsification, not a method failure: `pod-delete` churn produces no sustained endpoint outage, so the availability layer has little reproducible between-condition signal to estimate (that face is fault-matched to node-drain — and indeed, recomputed under node-drain the availability layer carries large, well-defined signal, though its ICC is 1.0 *by construction*, §5.8). The mechanism layer is the headline reliability result — ICC 0.994 versus the naive aggregate's 0.066: the layered scorecard's conntrack-reconvergence sub-score is far more test-retest reliable than a single aggregate score. The verdict is robust to the analysis's taint-handling choice (the conjunction fails either way). A single-campaign 0.994 with a wide-ish lower CI bound is strong but is re-evaluated across environments before being over-read (Figure 5.5).

5.6 Confirmatory family — Holm capstone

The four confirmatory hypotheses are corrected together by Holm at $\alpha = 0.05$, each member's family-input p read verbatim from its own analysis driver.

hyp	primary test	input p	Holm-adj p	supported?
H1	dose-response (Page's L)	0.0002	**0.0008**	no (effect < SESOI)
H2	placement + DNS	0.98875	0.98875	no (placement reversed)
H3	replication rescue	0.0065	**0.0195**	no (rescue margin unmet)
H5	layered scorecard ICC	0.2501	0.5002	no (availability ICC < 0.5)

Family verdict: no confirmatory hypothesis is supported. Two primaries (H1's trend, H3's interaction) survive Holm as statistically significant, but Holm-significance is necessary, not sufficient: each member also carries a registered effect-size / reliability bar the p-value cannot speak to, and every member fails its bar or its significance. The consistent, positive thread across all three campaigns is the *mechanism*: the UDP-conntrack reconvergence signature is real, individually significant in every campaign (H2's placement effect, H3's interaction, H5's mechanism ICC 0.994), and movable by placement and by the DNS intervention. What does *not* hold, at the pre-registered bar, is that this mechanism translates into the user-visible placement, availability, and dose-response advantages the hypotheses predicted. The discussion (§6.1) develops this as the study's central finding: a real mechanism, rigorously bounded in reach.

5.7 External validity — a second workload

Motivation. The single largest threat to these results (§7.2) is that they come from one workload. As an exploratory check — *outside* the frozen confirmatory family, so no multiplicity correction is owed — the two placement-bearing studies (H1 dose-response and H3 replication rescue) were replicated on a second, deliberately different application: DeathStarBench hotelReservation, a deep frontend fan-out with per-service datastore pairs and Go gRPC over Consul service discovery, against online-boutique's flatter topology and ClusterIP routing.

Campaign. The protocols were identical: 8 H1 complete-block sessions (`pod-delete` on the search path, r=1, five f-levels × 3 iterations) and 24 H3 sessions (`node-drain`; cells r1-packed, r3-packed, r3-anti-affine × 8). All 32 sessions were `doctor -strict` clean with pristine provenance; 1 of 144 iterations was excluded on an `app_ready_timeout` taint.[1]

Result — both arms corroborate online-boutique.

study	registered bar	hotelReservation outcome
H1 dose-response	monotone east-west p95 increase, ≥ 15 % SESOI	Page's L $p = 0.99$ — **no increase**
H3 replication rescue	r×mode interaction + rescue ≥ margins	`CONJUNCTION = False`

H1 on hotelReservation shows *no* dose-response of the east-west tail (Figure 5.6a): the registered one-sided trend test is non-significant ($p = 0.99$) and the per-level medians (9.34/6.44/6.32/7.10/5.40 ms) sit in a tight band with, if anything, a mild *decrease*. This is the same negative reading as online-boutique's sub-SESOI H1 (§5.1), slightly strengthened — no increase at all on the deeper topology. H3 reaches online-boutique's top-line verdict (`CONJUNCTION = False`; Figure 5.6b) by the same route — a *significant* r×mode interaction on both co-primaries ($p \approx 0$) with anti-affine r=3 the directionally best arm on every measure (trough recovery 15 s versus r=1's 45 s, during-chaos user-error 0.0 versus 0.21) and the packing controls holding — though the conjunction fails more broadly here: where online-boutique cleared the user-error margin and failed only

[1] Measuring hotelReservation required general ChaosProbe fixes, not changes to the experimental design or the system under test: a wget-capable readiness-probe pod, a static-topology fallback for the cross-node fraction on Consul/gRPC services that expose no service-address environment dependencies, and sustained warm-up load through the readiness gate. A prior scoping attempt mis-attributed the resulting gate timeouts to the workload's gRPC keepalive recovery; the cause was these tooling gaps, and once closed the campaign ran clean.

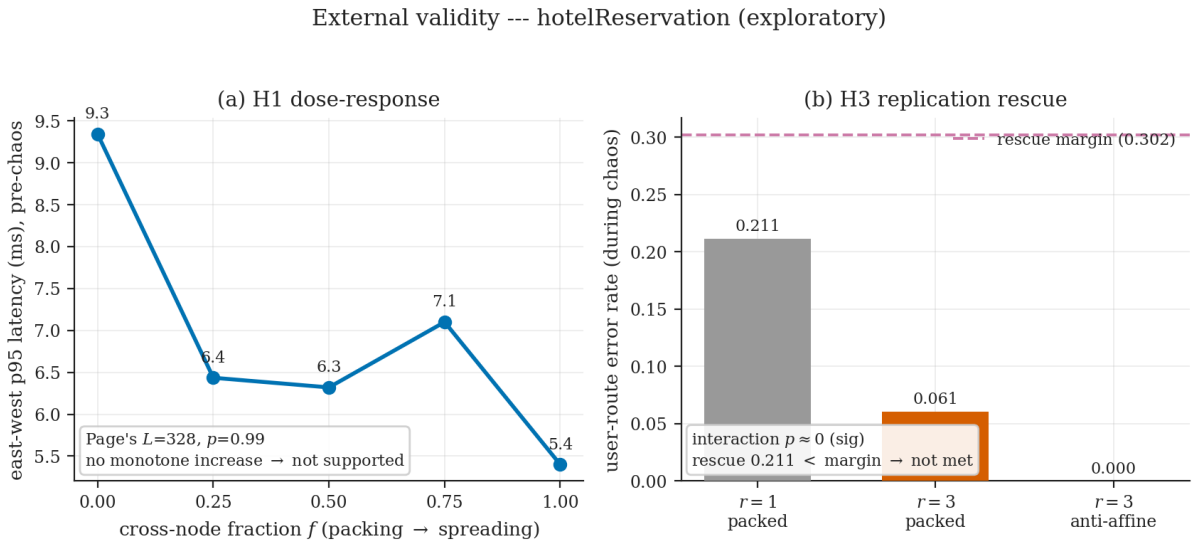

Figure 5.6: **External validity on hotelReservation** (exploratory, outside the Holm family). **(a)** H1 dose-response: east-west p95 latency shows no monotone increase in the cross-node fraction f (Page's L $p = 0.99$), a tight ∼5–9 ms band trending mildly downward — not supported. **(b)** H3 replication rescue: anti-affine r=3 drives the during-chaos user-route error to zero (from r=1's 0.21) with a significant r×mode interaction, but the rescue stays below the registered 0.302 margin — not met. Both arms corroborate online-boutique.

on trough depth (§5.3), hotelReservation clears *neither* margin (depth 0.044 < 0.0526, error 0.212 < 0.302).

Reading. The cross-workload picture mirrors the within-workload one (§5.6): the *mechanism* — spreading replicas across failure domains shrinks the node-drain blast radius and speeds recovery — is robust and visible on both applications, while the *strong placement and margin-clearing claims* hold on neither. That a second, structurally different workload reproduces both the below-SESOI dose-response and the significant-but-margin-short rescue is the strongest available evidence that the study's central finding — a real mechanism, rigorously bounded in reach — is not an artifact of one application's topology. This arm is exploratory and its verdicts are not folded into the Holm family; its full numbers, per-session provenance, and frozen integrity manifest are recorded in the committed C1/C2 hotelReservation campaign reports, with the raw dataset archived and available on request.

5.8 Design-corrected re-analysis of the availability axis

Motivation. Three of the availability-side tests are *construction-limited* in the confirmatory regime, all for one reason: `pod-delete` at r=1 removes a service's only replica, so the availability trough is ≈ 1 pod for *every* placement and the availability axis cannot move. This makes H3's trough-depth co-primary un-passable (its 1-pod margin equals the realized r=1 depth, §5.3), H4's availability face degenerate (constant, §5.4), and H5's availability-sub-score ICC a measurement of noise (§5.5). As an exploratory, pre-declared follow-up — *outside* the frozen Holm family — the three are recomputed under `node-drain`, whose blast radius is the set of services co-located with the drained node and therefore varies with placement. Criteria were fixed before the new data were examined (a design-corrected analogue of the freeze-before-analysis rule). The new campaign is C4: a node-drain dose-response, 8 complete-block sessions ($f \in \{0, 0.25, 0.5, 0.75, 1.0\}$, r=1), all `doctor -strict` clean.

The availability axis is now live (corrects H4). The node-drain trough depth varies

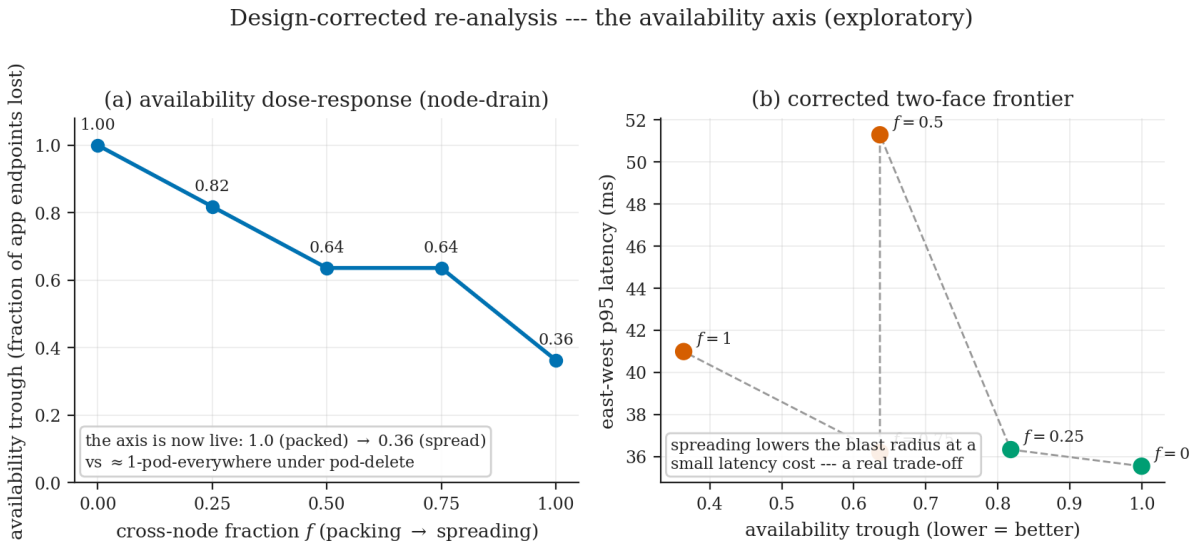

Figure 5.7: **Design-corrected re-analysis — the availability axis made live** (C4, `node-drain`, exploratory). **(a)** The availability trough now varies monotonically with placement, 1.0 (packed) → 0.36 (spread), against the ≈1-pod-everywhere degeneracy of `pod-delete` that made H4's availability face flat and H5's availability ICC a measurement of noise. **(b)** The corrected two-face frontier: spreading lowers the blast radius at a small east-west latency cost — a real trade-off (f=0.5 is a noise-dominated latency outlier). The arm is exploratory and outside the Holm family.

strongly and monotonically with placement, and a genuine latency×availability trade-off appears:

f (packing → spreading)	avail. trough (median)	east-west p95 (ms)
f=0 (packed)	**1.00** (all endpoints lost)	35.5
f=0.25	0.82	36.3
f=0.50	0.64	51.3
f=0.75	0.64	36.2
f=1 (spread)	**0.36**	41.0

The availability face spans 1.0 → 0.36 (against the degenerate ≈ 1-pod-everywhere of `pod-delete`): spreading buys a large availability gain at a small (∼35→41 ms, sub-SESOI) latency cost. The latency face rises mildly and monotonically except at f=0.5, whose 51.3 ms median is a noise-dominated outlier (two of eight sessions at ∼78 ms); read with that caveat the curve is monotone. The corrected frontier is non-degenerate and spreading is the availability-optimal region (at a small latency cost — so not Pareto-dominant, but a real trade-off) — replacing H4's degenerate "all five non-dominated" (Figure 5.7).

The availability layer carries reliable signal (corrects H5). Computed on the node-drain sessions, the availability sub-score's test-retest ICC = 1.0 versus 0.180 under `pod-delete` — the 0.180 was *absent signal*, not an unreliable scorecard. The 1.0 is, however, 1.0 *by construction*: the fraction-solver placement is deterministic, so every session's blast radius is identical (between-session variance is identically zero) and the ICC carries no test-retest information. The honest claim is therefore the weaker one — under a fault that produces an outage the availability layer has large, well-defined signal — not a noise-limited reliability estimate.

The replication-rescue effect is real (corrects H3's artifact). Re-analysing the C2 node-drain data, anti-affine r=3 *halves* the trough depth (0.091 → 0.046; r×mode interaction p=0.0065) and drives the user-route error to *zero* (0.63 → 0.0; $p \approx 0$). H3's registered

`CONJUNCTION = False` was an artifact of the 1-pod margin (which demanded a $> 100\,\%$ depth reduction); with it removed, the rescue is real and significant on both faces. On the unrounded medians the depth reduction is *exactly* $50\,\%$ (r=1 loses one pod, anti-affine half a pod-equivalent), so it meets the pre-declared range-relative bar — but knife-edge: at n=8 the anti-affine session depths are bimodal, so the median landing on the half-pod is a coincidence and the bar is not robustly cleared. The substantive, robust evidence is therefore the significant interaction and the user-error elimination, not the threshold itself.

Reading. This does not re-open the frozen confirmatory verdicts; it shows that H3's *not-supported* and H4's *degenerate* results were partly *construction artifacts* whose underlying effects are real once availability can move, and that H5's reliability gap was a fault-regime mismatch. The corrected arm is exploratory; its numbers, the reproducible driver (`scripts/design_fix_analysis.py`), and the frozen pre-analysis manifest (`c4-nodedrain-manifest.sha256`) are in the committed design-fix report, with the C4 raw data archived and available on request. A pre-registered confirmatory re-run that would elevate this corrected design from exploratory to confirmatory is specified in `AX-PREREGISTRATION-DRAFT.md` and was in calibration at the time of writing (§8.2).

5.9 H6 — direction transfer (not attempted)

H6 (an exploratory, non-family secondary on iptables-mode direction transfer) was **not attempted**: the cluster ran the IPVS proxier rather than the iptables mode the hypothesis presumes, so the comparison was de-scoped before collection (recorded in the deviations log). It supports no claim and is reported here only for completeness of the registered set.

Chapter 6

Discussion

6.1 A real mechanism, bounded in reach

The single thread that runs through all three campaigns is the *mechanism*: placement reproducibly moves a UDP-conntrack reconvergence signature, and that effect is individually significant every time it is measured — it is placement-dependent under churn (H2), it interacts with replication under node-drain (H3), and its scorecard sub-score is the one layer with near-perfect test-retest reliability (H5, ICC 0.994). What does *not* hold, under pre-registered and Holm-corrected testing, is that this mechanism translates into the user-visible placement, availability, and dose-response advantages the hypotheses predicted: no confirmatory hypothesis clears its registered bar (§5.6). The contribution is therefore a real mechanism whose reach is rigorously bounded, not a placement prescription.

Once `pod-delete` at r=1 is understood as a *churn* fault, the bound is the expected shape rather than a surprise. The injected event is the disappearance and recreation of a service's only replica; during the kill window the service is simply gone, and every dependent request fails identically whether the surrounding services are packed on one node or spread across many, because the user-visible outcome is governed by availability dynamics — the deletion-to-ready cycle of the single replica — not by the topology around it. What placement *does* govern is how much networking state the churn invalidates, which is exactly where H2's effect lives: at the kernel/network layer, not the user layer. The dose-response (H1) is the same story quantitatively — a monotone, Holm-significant trend in the east-west tail that is nonetheless only +13.35 % across the full cross-node range, below the 15 % SESOI: detectable, but too small to be the actionable placement lever the hypothesis sought.

The bound is regime-specific in a diagnosable way, and the failure modes of a single aggregate score compound rather than cancel. Under churn the score is **noisy** — the availability layer it leans on has little reproducible between-condition signal, which is exactly why the scorecard's availability sub-score is unreliable (H5, ICC 0.180 vs the mechanism layer's 0.994). Under node-drain it is **absent** — the drain takes down the infrastructure the Litmus probes depend on, every probe returns `Unknown`, and the score is undefined just when the availability outcome is most dramatic, which is why H3 measures availability from EndpointSlice troughs instead. A single-number instrument would need to be reliable in none of these ways; the layered scorecard's value (H5) is precisely that it separates the layer that *does* carry reproducible signal (mechanism) from the one that does not (availability) rather than averaging them into one unreliable number.

Even H3's one clearly-directional result — anti-affine replication fully rescuing the user-route error rate under node-drain (0.632 → 0.0) — does not lift the hypothesis over its registered bar, because the co-primary trough-depth criterion could not adjudicate rescue on this placement (the r=1 depth coincides with the one-pod margin by construction). That is a measurement-design limit surfaced honestly, not evidence against rescue; it is recorded as a lesson for a future pre-registration rather than retuned after the fact. The consistent picture across H1–H5 is the same:

where the mechanism is measured directly it is real and reproducible; where the hypotheses asked whether it reaches a user-visible, practically-significant outcome, the pre-registered answer is no.

6.2 The conntrack mechanism: attribution with protocol scoping

The conntrack-drop signature maps onto the reconvergence window the Kubernetes network-programming SLO defines, but the attribution must be protocol-scoped. Upstream maintainers document kube-proxy's *active* conntrack flush on endpoint churn as **UDP-only** (kubernetes/kubernetes #48370, #108523, #126130; TCP entries are deliberately never actively flushed — #100698, #104098). The application's east-west traffic is gRPC/**TCP**, so two paths plausibly contribute to the observed drop: kernel-side teardown of TCP flows traversing the killed pod (connection close transitions entries into short-timeout conntrack states — `close` 10 s, `close_wait` 60 s, `fin_wait`/`time_wait` 120 s, versus five days for `ESTABLISHED` — so FIN/RST teardown expires entries within $\leq$ 120 s of the kill, with no kube-proxy involvement), and the UDP/DNS pool that kube-proxy's documented cleanup acts on, which churns as connection turnover drives DNS re-resolution.

The DNS intervention (H2 arm b) settles which component placement controls. Enabling the NodeLocal DNSCache shrinks the during-churn drop by a 78 % median under the spread placement and *also* collapses it to ≈ 0 under the packed placement. The registered secondary predicted that the packed arm, which has no cross-node edges, would show no such cache effect if the drop were specifically cross-node DNS; that prediction fails (packed's effect is large), which pins the drop on the *general* UDP/DNS pool rather than cross-node resolution and removes the drop under *both* placements. The drop is therefore dominated by the DNS/UDP pool, exactly the traffic class the documented UDP-only cleanup targets. This also explains the *reversed* placement direction (H2 arm a): the registered prediction was that spreading dependencies across nodes would drive more cross-node DNS churn and thus a larger drop, but the data shows the opposite — packed exceeds spread in all 7 cache-off pairs. Co-locating a service's replicas on a single node *concentrates* the per-node conntrack churn when `pod-delete` hits that node, so the per-node drop the metric measures is larger under packing than under a placement that distributes the same churn across nodes. Placement does matter for the conntrack drop, significantly; the sign is simply the opposite of the hypothesis, and the DNS-cache result identifies the mechanism behind it.

What this does **not** establish is a full apportionment of the drop between TCP teardown and UDP cleanup from a single signature; that would need a steady-state, multi-iteration protocol-composition probe (§8.2). The defensible attribution is: the signature is real, reproducible, and placement-dependent; the UDP/DNS pool is the component whose size placement controls and whose removal (the DNS cache) removes the drop; and conntrack behaviour is pinned to the measured environment, since it changed materially across Kubernetes v1.31–v1.32 (§4.5).

6.3 Practical implications

- **Do not trust a single aggregate score to rank placements.** Its availability layer is unreliable under churn (H5: ICC 0.180) and undefined under node-drain, and a naive single-aggregate baseline is barely reliable at all (ICC 0.066). Audit a resilience score's reliability before using it to choose anything.
- **Measure the layer your fault class perturbs.** Churn shows up in conntrack reconvergence; node failure in EndpointSlice availability troughs. A single user-layer or score-layer probe misses the signal that the layered scorecard's mechanism sub-score captures reliably.

- **Placement advice is fault-class advice.** The mechanism is real but its user-visible reach is bounded and regime-specific; "spread for resilience" is meaningful only with the fault class and replication regime attached, and even its sign can invert (the conntrack drop is larger, not smaller, under packing here).
- **The DNS-cache intervention is the actionable lever.** It removes the during-churn conntrack drop under both placements — a concrete mitigation the experiments confirmed, independent of placement.

Each implication inherits its result's scope (§7.2). The deepest one is methodological: the value of pre-registering the hypotheses, fixing the SESOIs against a measured noise floor, and Holm-correcting the family is that it converts "we observed placement effects" into the more useful and more honest "here is exactly which effects survive a strict bar, and which do not." What is portable is the *method*: manipulate placement, measure at the layer the fault class perturbs, control the user layer with dependent-vs-control routes, gate every number on provenance, and audit the resilience score's reliability before trusting it to rank anything.

Chapter 7

Threats to validity & scope

This chapter states the boundary of every claim, in the standard vocabulary. **Construct validity** asks whether the measured quantities mean what the claims need: whether a single resilience score measures "resilience" at all is itself under test (H5 quantifies the layered scorecard's reliability against a naive single-aggregate baseline rather than assuming it), the UDP-conntrack drop is a *proxy* for reconvergence whose causal decomposition is explicitly pending (§6.2), and the `metricAvailability` bookkeeping exists because a missing metric silently read as zero would corrupt the construct it stands for. **Internal validity** asks whether observed differences are attributable to the manipulated variable: the threats are run-level drift, placement mismatch, and dirty provenance, and the defenses — the dependent-vs-control route split, within-run correlation, `placementMatchRates`, and the `doctor -strict` gate — are built into the design (§4.1, §4.3) rather than applied post hoc. **External validity** asks how far the results carry: a single small virtualized cluster, one workload, one Kubernetes version and kube-proxy mode, and a single-replica baseline bound the claims tightly, which is why §7.2 separates what generalizes (the method, the mechanism's direction) from what does not (the absolute values, anything multi-replica). **Conclusion validity** asks whether the statistics support the inferences: the design is *pre-registered* with SESOIs fixed against a measured A/A noise floor and the confirmatory family Holm-corrected, leaning on nonparametric tests, cluster-bootstrap intervals, and equivalence testing rather than significance hunting (§4.4). The tables below give the threat-by-threat detail.

7.1 Threats and defences

Threat	Why it matters	Defence
Single-replica baseline	At r=1, `pod-delete` removes the only instance, which can swamp topology effects on the availability layer.	Scope the churn claims to single-replica and layered measurement; the replication degree is itself a knob (H3 sweeps r=3 packed/anti-affine under node-drain).
Small virtualized cluster	Eight 2-vCPU/4-GiB KVM/libvirt workers may not generalize.	Claim bounded external validity; report *direction* and *mechanism*, not absolute latency values.
Version sensitivity	kube-proxy / conntrack behaviour evolves across releases (materially in v1.31–v1.32).	Archive exact Kubernetes (v1.28.6), runtime, kube-proxy mode/conntrack settings, and the analysis commit (`runMetadata` / manifests); present as a measurement study of a specific environment.
Placement mismatch	The solver may not realize the intended cross-node fraction.	The M1b gate verified all five f-levels within ± 0.05; `placementMatchRates` reported; mismatched iterations flagged or excluded.
Run-to-run drift	Iteration and session noise can dominate.	The A/A calibration block quantified the noise floor *before* freezing, and every SESOI/margin was fixed no smaller than that floor; independent single-commit sessions are the unit of analysis; runs are blocked/random effects; cluster-bootstrap CIs.
Dirty provenance	Untracked files / missing metadata undermine credibility.	Never quote results from runs failing `doctor -strict`; an early, weakly-provenanced pilot whose striking number did not survive strict re-runs is the institutionalized lesson behind the gate and the manifest dirty flag.
Metric-availability gaps	Missing PromQL queries can manufacture fake zeros.	`metricAvailability` distinguishes "not collected" from "collected zero" (e.g. kube-proxy programming latency is recorded as *uncollected* here, anchoring the mechanism only conceptually).
Overclaiming causality	Run-level slowness can confound correlations.	Dependent-vs-control routes + within-run correlation; TOST for equivalence claims; causal language reserved for the manipulated variable (placement) and the controlled DNS intervention.
Outcome-aware deviation	One deviation (D-2026-06-14-02, withdrawing the registered D3 UDP-slope taint from the C1 H1/H5 latency analyses) was decided *after* observing it made H1 unrunnable — the study's only non-blind deviation.	Disclosed in the H1 result (§5.1); justified on protocol-independent grounds (the band gates DNS/UDP, a different protocol than the latency outcome, and discards the cleanest levels); the taint-on sensitivity run is available, and H5's verdict is shown robust to the choice while H1's dependence is stated explicitly.

Two threats deserve a sentence beyond the table. The single-replica baseline is not only a threat but a *scoping instrument*: it is what makes `pod-delete` a pure churn fault (at r=1 the outage is total by construction, so any churn signal must appear at a non-availability layer), which is precisely why the availability face is degenerate under that fault (§5.4, §5.5). And the run-to-run drift row is not an admission that the environment was too noisy: quantifying that noise and setting the SESOIs above it is the point of the A/A calibration block, so the design

exists because the drift was measured, not despite it.

7.2 What generalizes vs. what does not

Aspect	Generalizes?	Why / caveat
The *method* (placement-aware, cross-layer, pre-registered, provenance-gated chaos evaluation)	**Yes**	The framework and analysis discipline are reusable on any cluster/workload.
The *direction* of the placement effect on the conntrack drop (H2)	**Direction only**	Reproducible here (packed concentrates more per-node conntrack churn than spread under `pod-delete`, 7/7 pairs); absolute values are environment-specific, and the direction is the *opposite* of the registered prediction.
The DNS-cache intervention (H2 arm b) removing the conntrack drop	**Direction only**	The NodeLocal DNSCache shrinks the drop under both placements here; the magnitude is environment-specific.
Absolute metric values (latency, recovery s, conntrack entries)	**No**	Tied to a small virtualized 8-worker cluster.
Mechanism behaviour (conntrack, kube-proxy sync)	**Environment-contingent**	Depends on CNI, kube-proxy mode (ipvs here), kernel/conntrack settings — archived per run so the scope is explicit.
The layered-scorecard reliability finding (H5)	**This regime**	The mechanism sub-score is highly test-retest reliable (ICC 0.994) and the availability sub-score is not (0.180) under single-replica `pod-delete`; not asserted for node-drain-dominated or multi-replica regimes.
The dose-response of the east-west tail (H1)	**This regime, sub-SESOI**	Detected and Holm-significant but below the 15 % SESOI on this cluster; a detectable-but-negligible trend, not a transferable magnitude.
The placement findings across *workloads* (H1, H3)	**Corroborated on a 2nd workload**	Both replicated on DeathStarBench hotelReservation (a deep Consul/gRPC fan-out): same below-SESOI dose-response verdict (sub-SESOI on online-boutique, null on hotel) and the same `CONJUNCTION = False` with a significant-but-margin-short rescue (§5.7). The mechanism-without-margin pattern is not an artifact of one topology; second *cluster*/infrastructure still untested.
Anything about multi-replica / HA failover	**Partly (node-drain only)**	H3 reaches r=3 under node-drain; broader HA-failover behaviour is out of scope (§8.2).

The asymmetry of this table is deliberate. The portable row is the first: the measurement design, the pre-registered campaign protocol, and the analysis scripts transfer to any cluster and workload unchanged: a second-workload replication (§5.7) corroborates both placement studies, and a replication on different infrastructure is now the most valuable remaining follow-up (§8.2). Everything below degrades gracefully from "direction only" to "not in scope," and where a row says environment-contingent, the archived manifests make the contingency checkable rather than vague: a reader can compare their kube-proxy mode, conntrack settings, and Kubernetes version against the archived fingerprint instead of guessing.

7.3 Explicitly not claimed

The study declines several stronger claims the data could be mistaken to support: no universal "best" placement; no claim that the placement or availability *effects are disproven* (the confirmatory hypotheses are *not supported at their registered bars* — a bounded result, not a refutation, and the mechanism they concern is real and significant); no proven causality for the conntrack mechanism (mechanistic consistency, not controlled causal proof); no generalization to Kubernetes clusters broadly; no claim that any single score "identifies the best strategy"; and no unqualified "reproducible" (only the mechanism metrics reproduce, and only with the archived rerun package).

These exclusions are commitments, not disclaimers: each names a stronger claim the data could be misread to support and that we decline. In particular, "no confirmatory hypothesis was supported" is stated precisely — two primaries (H1's trend, H3's interaction) are Holm-significant but fail their registered effect-size bars, and the consistent positive thread is the mechanism, not the user-visible placement advantages the hypotheses predicted (§5.6). Where this document's prose and the registered scope could ever be read to disagree, the registered scope governs.

Chapter 8

Conclusion & future work

8.1 Conclusion

This thesis asked: **under which chaos fault classes does pod placement measurably affect mechanism-level behaviour and user-visible outcomes in a Kubernetes microservice application, and when do aggregate resilience scores obscure those effects?** Answered through a pre-registered, Holm-corrected confirmatory family, the result is a real mechanism whose reach is rigorously bounded.

The positive thread is the *mechanism*. Placement reproducibly moves a UDP-conntrack reconvergence signature, and that effect is significant in every campaign: it is placement-dependent under churn (H2), it interacts with replication under node-drain (H3's error face, where anti-affine replication fully rescues the user-route error rate), and it is the one scorecard layer with near-perfect test-retest reliability (H5, mechanism ICC 0.994 vs a naive aggregate's 0.066). A NodeLocal DNSCache intervention removes the conntrack drop under both placements — the one actionable lever the study confirmed.

The bound is what pre-registration makes precise. No confirmatory hypothesis clears its registered bar (§5.6): the dose-response is detectable but below the 15 % SESOI (H1); placement-dependence replicates with the *opposite* sign, packing concentrating more per-node conntrack churn than spreading, so the conjunction fails (H2); replication's rescue is clear on the error face but the trough-depth co-primary could not adjudicate it on this placement (H3); and the scorecard's availability layer is unreliable under churn (H5). The latency↔availability frontier is likewise degenerate on the availability face under pod-delete (H4) — both reflecting that pod-delete produces no sustained availability outage to discriminate on. Placement moves the mechanism; at the pre-registered bar, it does not move the user-visible placement, availability, and dose-response outcomes the hypotheses predicted.

The three contributions of §1.3 carry this answer within the boundary of Chapter 7. The empirical finding — a real, DNS-mediated, placement-dependent conntrack mechanism whose user-visible reach is bounded — is a statement about a single-replica baseline on one small virtualized v1.28.6/ipvs cluster (§7.2), never "spread is worse" or a best strategy. ChaosProbe and the freeze-before-analysis campaign protocol are the portable parts: any cluster and workload can rerun this design, and every number traces to an archived, hash-stamped run (Appendix A).

Reported as an in-development, iterative design study, this thesis also demonstrates the method correcting itself: a first design iteration's single-replica `pod-delete` could not move the availability axis, the limitation was diagnosed and the design modified to `node-drain` (§5.8), and the corrected design — exploratory here, with a pre-registered confirmatory re-run underway — recovers the latency↔availability trade-off the original could not express. Nothing from the first iteration is discarded; surfacing the construction limit and correcting it under the same pre-registered, provenance-gated discipline is the part of the methodology a thesis is meant to show.

The methodological point is the one we would have the field retain. Chaos evaluation of placement needs **layered measurement, pre-registration, and provenance-gated replication, not a single score**: each fault class left its placement signal in a specific layer, a one-number instrument's availability component was unreliable or undefined exactly where it mattered, and freezing the hypotheses and effect-size bars before collection is what turned "we observed placement effects" into the more honest "here is exactly which effects survive a strict bar, and which do not."

8.2 Future work

Beyond the second workload

The single largest threat (§7.2) — that all confirmatory results come from one workload — has been *partly addressed*: the two placement-bearing studies were replicated on DeathStarBench hotelReservation, a deep Consul/gRPC fan-out, and both arms corroborate online-boutique (§5.7) — a null dose-response and a significant-but-margin-short replication rescue — the strongest available evidence that the central finding is not an artifact of one topology. That replication is exploratory (a second workload, not a second cluster), so the highest-value next steps now lie elsewhere: replicating the family on *different infrastructure* (a managed cluster, a different kube-proxy mode, non-virtualized nodes), extending to *more* workloads and faults to map how far the mechanism-without-margin pattern holds, and the margin-construction fixes below that would let the depth and dose-response faces be tested on their merits.

Decomposing and extending the mechanism

- **Conntrack-drop apportionment.** The drop has both a kernel TCP teardown component and the placement-dependent UDP/DNS pool the cache removes (§6.2); a steady-state, multi-iteration protocol-composition probe would apportion the two, converting the study's most reproducible signal from mechanism-consistent to causally decomposed.
- **A rescue-margin the depth face can express.** H3's trough-depth co-primary could not adjudicate rescue because the registered absolute one-pod margin coincided with the r=1 dynamic range on this placement; a margin defined relative to the realized r=1 depth (or an integrated outage = depth $\times$ duration metric) would let a future pre-registration test the depth face on its merits.
- **Denser dose levels and other faults.** Finer cross-node-fraction steps around the detected (sub-SESOI) trend, and a fault that varies the availability face while retaining a latency face, would let the frontier (H4) realize the trade-off it is degenerate on here.

Environment and scope

Larger and bare-metal clusters, other CNIs and kube-proxy modes (iptables, nftables — the de-scoped direction-transfer comparison), multi-replica workloads beyond the node-drain regime H3 reaches, production traffic, and integrating the cross-node fraction into a scheduler as a scoring plugin. The common thread is the method: each of these runs on the same layered, pre-registered, provenance-gated protocol this thesis contributes, and each would sharpen — not merely extend — the boundary of where placement matters.

These external-validity steps cohere into a single *generalization extension*: a pre-registered second-environment direction-transfer arm (which subsumes the de-scoped iptables/nftables comparison), a fresh confirmatory second-workload arm, and the conntrack-composition probe above. It bears stating plainly that such an extension would *discharge* the standing threats by widening the tested set, not eliminate them — even fully run, the study stays bounded by

the environments and workloads it covers. Closing scope threats is a matter of disclosure and bounded extension, not a limitation-free end state.

Appendix A

Run provenance

Every confirmatory number in Chapter 5 traces to one of three pre-registered campaigns, each *frozen at a tagged commit before its analysis was written.* The campaigns share an environment fingerprint, recorded in every session's `runMetadata` and in each archive's `artifact-manifest.json` (written by `archive_run.py -strict`): Kubernetes **v1.28.6**, kube-proxy mode **ipvs**, containerd 1.7.11, Calico CNI, eight 2-vCPU/4-GiB workers, namespace `online-boutique`, git `dirty: false`, `provenanceWarnings: []`. The frozen pre-registration (commit `20097c1`), the SHA-256 integrity manifests, and the analysis code are committed to the project repository; its hypotheses, smallest effect sizes of interest, and per-cell sample sizes are also stated bar-first in Chapter 5. The raw per-run session archives are retained with the project materials and available on request.

A.1 Campaigns

campaign	**tests**	**design**	**sess.**	**data commit**
C1 dose-response	H1, H4, H5	complete-block $f \in \{0, 0.25, 0.5, 0.75, 1.0\}$, r=1, `pod-delete` churn; each session visits all five levels in seeded-random order, $5 \times 3 = 15$ iterations	8 (s01–s08)	`2ec934f`
C2 replication rescue	H3	between-subjects $r \in \{1, 3\}$ × mode $\in$ {packed, anti-affine}, three non-degenerate cells (r1-packed, r3-packed, r3-anti-affine) × 8, `node-drain` at f=0.5	24	`e533d5b`
C3 placement + DNS	H2	within-subjects $f \in \{0, 1\}$ at r=1, crossed with `dnsCache` $\in$ {off, on} (NodeLocal DNSCache via pod `dnsConfig`); 7 matched cache-off/cache-on pairs, `pod-delete` churn, 3 iterations per placement	14	`2409f35`

The **data commit** column records the commit each campaign's data was collected on (pinned in every session's `runMetadata`). C1 was collected on data commit `2ec934f`; its analysis runs on `main` at or after `7d6943e`, so the data commit predates the analysis tooling by design (deviation D-2026-06-14-01). C2 and C3 were collected and analyzed on a single strict-clean commit each.

A.2 Claims → campaigns

Claim (Chapter 5)	Campaign / analysis
H1 dose-response, +13.35 % < 15 % SESOI (§5.1)	C1 (8 sessions); Page's L trend test on per-session level medians
H2 placement-dependence (reversed) + DNS arm (§5.2)	C3 (7 cache-off + 7 cache-on sessions); paired drop contrast + within-spread cache comparison
H3 replication rescue under node-drain (§5.3)	C2 (24 sessions); ART-ANOVA $r \times$ mode interaction on trough depth and user-route error
H4 descriptive placement frontier (§5.4)	C1 (the five r=1 `pod-delete` placements; analysis-only, no additional runs)
H5 layered scorecard ICC, mechanism 0.994 vs naive aggregate 0.066 (§5.5)	C1 (condition-level test-retest reliability across the 8 sessions)
Family Holm capstone (§5.6)	H1, H2, H3, H5 corrected together; each member's family-input p read verbatim from its own driver
H6 direction transfer (§5.9)	not attempted — the cluster ran the ipvs proxier, not the iptables mode the hypothesis presumes; de-scoped before collection, no runs

A.3 Integrity

Each session directory carries `summary.json`, its per-condition raw JSON (`f-*.json`), `charts/`, and an `artifact-manifest.json` pinning run identity, git provenance, the SHA-256 of the exact scenario YAML injected, and a SHA-256 of every file in the session. Every campaign additionally carries a root `SHA256SUMS`; C2's and C3's raw checksums are pinned in the committed manifests `c2-roundrobin-manifest.sha256` and `c3-dns-manifest.sha256` (computed pre-analysis). All sessions passed `archive_run.py -strict` (clean tree, scenario hashes present, `runMetadata` present): C1 8/8, C2 24/24 (all accepted, untainted, 8 per cell), C3 14/14 (all accepted, untainted, 7 per cache mode). Per-session `summary.json` SHA-256 anchors are tabulated in each campaign's committed integrity manifest; representative anchors are C1 s01 `b2282cce...f2301456`, C2 #1 (r1-packed) `e4b3c50f...504f408`, and C3 #1 (cache-off) `83169d79...f7fc6a82`.

A.4 External-validity campaign (exploratory)

The second-workload replication (§5.7) is **exploratory and outside the frozen Holm family**; it is provenance-gated to the same bar as the confirmatory campaigns but tests no pre-registered hypothesis, so it is recorded separately here. Workload: DeathStarBench `hotelReservation` (namespace `hotel-reservation`), a deep frontend fan-out with per-service datastore pairs and Go gRPC over Consul service discovery, on the same eight-worker cluster.

arm	test	design	sess.	data commit
C1-hotel dose-response	H1	complete-block $f \in \{0, 0.25, 0.5, 0.75, 1.0\}$, r=1, `pod-delete` on the search path, 5×3 iterations	8	`bdf1ccb`
C2-hotel replication rescue	H3	three cells (r1-packed, r3-packed, r3-anti-affine) $\times 8$, `node-drain` at f=0.5, round-robin assignment	24	`bdf1ccb`

Both arms were collected and analyzed on `main` commit `bdf1ccb` (`git.dirty: false`). All 32 sessions are `doctor -strict` clean; 1 of the 144 C1/C2-hotel iterations (120 C1-hotel

`pod-delete` churn iterations + 24 C2-hotel `node-drain` sessions; the excluded one is an iteration of C1-hotel order-seed 8, collected after a deliberate pause/restart) carried an `app_ready_timeout` taint and is excluded by the registered healthy-only rule. The pre-analysis SHA-256 integrity manifest (`c1-c2-hotel-manifest.sha256`) is committed to the project repository; the raw data is archived and available on request. Measuring this workload required general tooling fixes (a wget-capable readiness-probe pod, a static-topology fallback for the cross-node fraction on Consul/gRPC services, and sustained warm-up load through the readiness gate), none of which alter the experimental design or the system under test.

A.5 Design-corrected re-analysis (exploratory)

The availability-axis re-analysis (§5.8) is likewise **exploratory and outside the Holm family**. Its corrected criteria were pre-declared (in `DESIGN-FIX-SCOPE.md`) before the new data were examined. New data: campaign **C4**, a `node-drain` dose-response on online-boutique — 8 complete-block sessions ($f \in \{0, 0.25, 0.5, 0.75, 1.0\}$, r=1, 5×3 iterations), `results/c4-nodedrain-dose/`, collected on `main` (`git.dirty: false`; sessions 1–2 commit `445c5bc`, 3–8 commit `71c545d` — identical run-path code, the intervening merge was thesis-docs only). All 8 sessions `doctor -strict` clean. The H3 correction re-uses the existing C2 node-drain data (no new run — the fault was the criterion). Pre-analysis manifest `c4-nodedrain-manifest.sha256` and the reproducible driver `scripts/design_fix_analysis.py` are committed to the project repository; the C4 raw data is archived and available on request. A confirmatory follow-up that elevates this corrected design from exploratory to pre-registered is specified in the project repository (`AX-PREREGISTRATION-DRAFT.md`) and was in calibration at the time of writing (§8.2).

Bibliography

Amazon Web Services. Reducing the scope of impact with cell-based architecture. AWS Well-Architected whitepaper. URL https://docs.aws.amazon.com/wellarchitected/latest/reducing-scope-of-impact-with-cell-based-architecture/reducing-scope-of-impact-with-cell-based-architecture.html. Accessed 2026-06-11.

Awad et al. Chaos experiments in microservice architectures: A systematic literature review. *Journal of Systems and Software*, 2025. URL https://www.sciencedirect.com/science/article/abs/pii/S092054892500145X.

M. Barletta, M. Cinque, C. Di Martino, Z. T. Kalbarczyk, and R. K. Iyer. Mutiny! How does Kubernetes fail, and what can we do about it? In *54th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2024)*, 2024. URL https://arxiv.org/abs/2404.11169.

A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016. doi: 10.1109/MS.2016.60. URL https://dl.acm.org/doi/abs/10.1109/MS.2016.60.

A. Benavoli, G. Corani, J. Demšar, and M. Zaffalon. Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis. *Journal of Machine Learning Research*, 18(77), 2017. URL https://jmlr.org/papers/v18/16-305.html.

B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes. *ACM Queue*, 14, May 2016. doi: 10.1145/2890784. URL https://dl.acm.org/doi/10.1145/2890784. Also: Communications of the ACM 59(5), May 2016.

Cast. Cast: Automated resilience testing via online traffic record-and-replay with fault injection. ICSE 2026 SEIP track; arXiv:2602.00972, 2026. URL https://arxiv.org/html/2602.00972v1. Accessed 2026-06-11.

Chaos Engineering MLR. Chaos engineering: A multi-vocal literature review. arXiv:2412.01416; ACM Computing Surveys, DOI 10.1145/3777375, 2024. URL https://arxiv.org/abs/2412.01416. Accessed 2026-06-11.

E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 2017)*, 2017. doi: 10.1145/3132747.3132772. URL https://dl.acm.org/doi/10.1145/3132747.3132772.

J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, February 2013. doi: 10.1145/2408776.2408794. URL https://dl.acm.org/doi/10.1145/2408776.2408794.

C. Delimitrou and C. Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2014)*, Salt Lake City, UT, 2014. URL https://www.csl.cornell.edu/~delimitrou/papers/2014.asplos.quasar.pdf.

Y. Gan et al. An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2019)*, 2019. doi: 10.1145/3297858.3304013. URL https://dl.acm.org/doi/10.1145/3297858.3304013.

P. Garefalakis, K. Karanasos, P. Pietzuch, A. Suresh, and S. Rao. Medea: Scheduling of long running applications in shared production clusters. In *Proceedings of the Thirteenth European Conference on Computer Systems (EuroSys 2018)*, 2018. doi: 10.1145/3190508.3190549. URL https://dl.acm.org/doi/abs/10.1145/3190508.3190549.

Google Cloud Platform. microservices-demo: Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC. GitHub repository. URL https://github.com/GoogleCloudPlatform/microservices-demo. Accessed 2026-06-11.

Hagedoorn et al. An empirical study on how architectural topology affects microservice performance and energy usage. arXiv:2604.00080, 2026. URL https://arxiv.org/abs/2604.00080. Accessed 2026-06-11.

T. Hoefler and R. Belli. Scientific benchmarking of parallel computing systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2015)*, 2015. doi: 10.1145/2807591.2807644. URL https://dl.acm.org/doi/10.1145/2807591.2807644.

G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. URL https://www.cs.utexas.edu/~pingali/CS395T/2009fa/papers/metis.pdf. Also: Multilevel k-way Partitioning Scheme for Irregular Graphs, Journal of Parallel and Distributed Computing, 48, 1998, pp. 96–129.

D. Kikuta, H. Ikeuchi, and K. Tajiri. ChaosEater: LLM-powered fully automated chaos engineering. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025), NIER track*, pages 3861–3865, 2025. doi: 10.1109/ASE63991.2025.00331. URL https://arxiv.org/abs/2511.07865.

Kubernetes Community. kube-proxy netlink conntrack reconciler (pkg/proxy/conntrack/cleanup.go). Source file, kubernetes/kubernetes, a. URL https://github.com/kubernetes/kubernetes/blob/master/pkg/proxy/conntrack/cleanup.go. Accessed 2026-06-11.

Kubernetes Community. conntrack entries not cleared when switching service endpoints (kubernetes/kubernetes issue #100698). GitHub issue, kubernetes/kubernetes, b. URL https://github.com/kubernetes/kubernetes/issues/100698. Accessed 2026-06-11.

Kubernetes Community. Kubernetes doesn't clear conntrack entry for TCP (kubernetes/kubernetes issue #104098). GitHub issue, kubernetes/kubernetes, c. URL https://github.com/kubernetes/kubernetes/issues/104098. Accessed 2026-06-11.

Kubernetes Community. conntrack entries removed on service/endpoint deletion are UDP & SCTP (kubernetes/kubernetes issue #108523). GitHub issue, kubernetes/kubernetes, d. URL https://github.com/kubernetes/kubernetes/issues/108523. Accessed 2026-06-11.

Kubernetes Community. Connections stuck in conntrack when externalTrafficPolicy is "local" (kubernetes/kubernetes issue #113203). GitHub issue, kubernetes/kubernetes, e. URL https://github.com/kubernetes/kubernetes/issues/113203. Accessed 2026-06-11.

Kubernetes Community. kube-proxy exec-based conntrack cleaner logs (kubernetes/kubernetes issue #125467). GitHub issue, kubernetes/kubernetes, f. URL https://github.com/kubernetes/kubernetes/issues/125467. Accessed 2026-06-11.

Kubernetes Community. conntrack reconciler; "UDP is the only protocol that requires this" (kubernetes/kubernetes issue #126130). GitHub issue, kubernetes/kubernetes, g. URL https://github.com/kubernetes/kubernetes/issues/126130. Accessed 2026-06-11.

Kubernetes Community. v1.32 netlink-reconciler regression: any UDP-port change triggered a full cleanup pass (kubernetes/kubernetes issue #129982, since fixed). GitHub issue, kubernetes/kubernetes, h. URL https://github.com/kubernetes/kubernetes/issues/129982. Accessed 2026-06-11.

Kubernetes Community. Frequent churn between EndpointSlice objects (kubernetes/kubernetes issue #133474). GitHub issue, kubernetes/kubernetes, i. URL https://github.com/kubernetes/kubernetes/issues/133474. Accessed 2026-06-11.

Kubernetes Community. Flush stale UDP conntrack entries when endpoints change (kubernetes/kubernetes issue #48370). GitHub issue, kubernetes/kubernetes, j. URL https://github.com/kubernetes/kubernetes/issues/48370. Accessed 2026-06-11.

Kubernetes Community. Investigate issues with Network Programming Latency SLO (kubernetes/kubernetes issue #82378). GitHub issue, kubernetes/kubernetes, k. URL https://github.com/kubernetes/kubernetes/issues/82378. Accessed 2026-06-11.

Kubernetes Community. conntrack entries not cleared when pod moves (kubernetes/kubernetes issue #93143). GitHub issue, kubernetes/kubernetes, l. URL https://github.com/kubernetes/kubernetes/issues/93143. Accessed 2026-06-11.

Kubernetes Community. Network programming latency SLO. sig-scalability, kubernetes/community repository, m. URL https://github.com/kubernetes/community/blob/master/sig-scalability/slos/network_latency.md. Accessed 2026-06-11.

Kubernetes Community. DNS intermittent delays of 5s (kubernetes/kubernetes issue #56903). GitHub issue, kubernetes/kubernetes, 2017–2019, 2017. URL https://github.com/kubernetes/kubernetes/issues/56903. Accessed 2026-06-11.

Kubernetes SIG Scheduling. KEP-895: Pod topology spread. Kubernetes Enhancement Proposal 895, sig-scheduling. URL https://github.com/kubernetes/enhancements/tree/master/keps/sig-scheduling/895-pod-topology-spread. Accessed 2026-06-11.

D. Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4), 2017. URL https://journals.sagepub.com/doi/10.1177/1948550617697177.

D. Lakens, A. M. Scheel, and P. M. Isager. Equivalence testing for psychological research. *Advances in Methods and Practices in Psychological Science*, 1(2), 2018. URL https://journals.sagepub.com/doi/10.1177/2515245918770963.

LitmusChaos contributors. litmus-go: node-memory-hog chaos library (calculateMemoryConsumption). Source file, litmuschaos/litmus-go. URL https://github.com/litmuschaos/litmus-go/blob/master/chaoslib/litmus/node-memory-hog/lib/node-memory-hog.go. Accessed 2026-06-11.

Liu et al. Resilience evaluation of Kubernetes in cloud-edge environments via failure injection. arXiv:2507.16109, 2025. URL https://arxiv.org/abs/2507.16109. Accessed 2026-06-11.

A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci. Taming performance variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2018)*, pages 409–425, 2018. URL https://www.usenix.org/conference/osdi18/presentation/maricq.

J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa. Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2011)*, pages 248–259, 2011. URL https://www.cs.virginia.edu/~skadron/Papers/mars_micro2011.pdf.

T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2009)*, 2009. URL https://www.semanticscholar.org/paper/3886c40229b3de318de668e0c0f4202079eb6f55.

Netflix Technology Blog. The Netflix simian army. Netflix Technology Blog; open-sourced as Netflix/SimianArmy (later Netflix/chaosmonkey), 2011. URL https://netflixtechblog.com/the-netflix-simian-army-16e57fbab116. Accessed 2026-06-11.

K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica. Sparrow: Distributed, low latency scheduling. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 2013)*, 2013. doi: 10.1145/2517349.2522716. URL https://dl.acm.org/doi/10.1145/2517349.2522716.

Priority Matters. Priority matters: Optimising Kubernetes clusters usage with constraint-based pod packing. arXiv:2511.08373, 2025. URL https://arxiv.org/abs/2511.08373. Accessed 2026-06-11.

M. Pumputis. Racy conntrack and DNS lookup timeouts. Weaveworks blog (site defunct — use an archive.org link). Accessed via archive.org, 2026-06-11.

Resilient Microservices SLR. Resilient microservices: A systematic review of recovery patterns, strategies, and evaluation frameworks. arXiv:2512.16959, 2025. URL https://arxiv.org/abs/2512.16959. Accessed 2026-06-11.

D. J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15, 1987. doi: 10.1007/BF01068419. URL https://doi.org/10.1007/BF01068419.

The Linux kernel development community. Netfilter conntrack sysctl reference. kernel.org documentation. URL https://docs.kernel.org/networking/nf_conntrack-sysctl.html. Accessed 2026-06-11.

TraDE. TraDE: Network and traffic-aware adaptive scheduling for microservices under dynamics. arXiv:2411.05323, 2024. URL https://arxiv.org/abs/2411.05323. Accessed 2026-06-11.

A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys 2015)*, Bordeaux, France, 2015. doi: 10.1145/2741948.2741964. URL https://dl.acm.org/doi/10.1145/2741948.2741964.

P. Versockas. A technique to monitor Kubernetes controller latency. Practitioner blog, povilasv.me. URL https://povilasv.me/a-technique-to-monitor-kubernetes-controller-latency/. Accessed 2026-06-11.

K. Wojciechowski et al. NetMARKS: Network metrics-AwaRe Kubernetes scheduler powered by service mesh. In *IEEE INFOCOM 2021 — IEEE Conference on Computer Communications*, 2021. doi: 10.1109/INFOCOM42981.2021.9488670. URL https://ieeexplore.ieee.org/document/9488670/.

XING Engineering. A reason for unexplained connection timeouts on Kubernetes/Docker. XING engineering blog. URL https://tech.xing.com/a-reason-for-unexplained-connection-timeouts-on-kubernetes-docker-abd041cf7e02. Accessed 2026-06-11.

Yang et al. Network-aware reliability modeling and optimization for microservice placement. arXiv:2405.18001, 2024a. URL https://arxiv.org/abs/2405.18001. Accessed 2026-06-11.

T. Yang, C. Lee, J. Shen, Y. Su, C. Feng, Y. Yang, and M. R. Lyu. MicroRes: Versatile resilience profiling in microservices via degradation dissemination indexing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*, Vienna, Austria, 2024b. doi: 10.1145/3650212.3652131. URL https://arxiv.org/abs/2212.12850.

Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou. Sinan: ML-based and QoS-aware resource management for cloud microservices. In *Proceedings of the 26th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2021)*, 2021. doi: 10.1145/3445814.3446693. URL https://dl.acm.org/doi/10.1145/3445814.3446693.